TRƯỜNG ĐẠI HỌC PHENIKAA

KHOA KHOA HỌC MÁY TÍNH

BÁO CÁO ĐỒ ÁN CUỐI KỲ

MÔN THUẬT TOÁN ỨNG DỤNG

HỆ THỐNG TỐI ƯU LỘ TRÌNH VÀ MÔ PHỎNG TRUY TÌM TRÊN MẠNG ĐƯỜNG HÀ NỘI

Giảng viên hướng dẫn : TS. Trần Đình Tân

Nhóm sinh viên : Hoàng Tiến Đạt – 23010864

Mầu Danh Chiến – 23017270

Phạm Văn Tuấn Kiệt – 23010881

Hà Nội, tháng 10 năm 2026

TRƯỜNG ĐẠI HỌC PHENIKAA

KHOA KHOA HỌC MÁY TÍNH

BÌA PHỤ BÁO CÁO ĐỒ ÁN CUỐI KỲ

MÔN THUẬT TOÁN ỨNG DỤNG

HỆ THỐNG TỐI ƯU LỘ TRÌNH VÀ MÔ PHỎNG TRUY TÌM TRÊN MẠNG ĐƯỜNG HÀ NỘI

Giảng viên hướng dẫn : TS. Trần Đình Tân

Nhóm sinh viên : Hoàng Tiến Đạt – 23010864

Mầu Danh Chiến – 23017270

Phạm Văn Tuấn Kiệt – 23010881

Hà Nội, tháng 10 năm 2026

LỜI CẢM ƠN

Nhóm chúng em xin trân trọng cảm ơn **TS. Trần Đình Tân** đã hướng dẫn và giảng dạy học phần Thuật toán ứng dụng. Kiến thức về mô hình hóa bài toán, lựa chọn thuật toán, phân tích độ phức tạp và kiểm chứng thực nghiệm là nền tảng để nhóm xây dựng đồ án này.

Nhóm cũng xin cảm ơn Khoa Khoa học máy tính, Trường Đại học Phenikaa đã tạo môi trường học tập và cung cấp các điều kiện cần thiết cho quá trình thực hiện. Các thành viên trong nhóm đã phối hợp trong quá trình phân tích, lập trình, kiểm thử và hoàn thiện sản phẩm.

Mặc dù đã cố gắng kiểm tra kỹ, báo cáo và phần mềm vẫn có thể còn thiếu sót. Nhóm mong nhận được ý kiến góp ý của giảng viên để tiếp tục hoàn thiện hệ thống.

Hà Nội, tháng 10 năm 2026

Nhóm sinh viên thực hiện

LỜI CAM ĐOAN

Nhóm sinh viên thực hiện xin cam đoan rằng báo cáo đồ án cuối kỳ môn *Thuật toán ứng dụng* với đề tài "Hệ thống tối ưu lộ trình và mô phỏng truy tìm trên mạng đường Hà Nội" là công trình do chính nhóm thực hiện dưới sự hướng dẫn của **TS. Trần Đình Tân**.

Cụ thể, nhóm cam đoan:

- Toàn bộ mã nguồn trong thư mục `src/`, `tests/` và `scripts/` do nhóm tự viết. Các thuật toán Dijkstra, Held–Karp, nearest-neighbor, 2-opt, luật trò chơi và ba chiến thuật truy tìm được triển khai trực tiếp, không gọi hàm giải sẵn của thư viện. NetworkX chỉ được dùng cho cấu trúc dữ liệu đồ thị và làm oracle đối chiếu trong kiểm thử.
- Mọi số liệu thực nghiệm trong Chương 5 là kết quả đo thật trên máy của nhóm, được sinh bằng các lệnh ghi trong Phụ lục A.2 và lưu kèm dưới dạng JSON trong thư mục `output/`. Không có số liệu nào được ước lượng, làm tròn có lợi hoặc chép từ nguồn khác.
- Dữ liệu bản đồ có nguồn gốc từ OpenStreetMap và được sử dụng theo giấy phép ODbL; phần xử lý, trích xuất và đóng gói do nhóm thực hiện.
- Các kiến thức, công thức và kết quả tham khảo từ tài liệu bên ngoài đều được trích dẫn đầy đủ trong phần Tài liệu tham khảo.

Nhóm xin chịu hoàn toàn trách nhiệm trước Khoa và Nhà trường về tính trung thực của nội dung báo cáo này.

Hà Nội, tháng 10 năm 2026

Nhóm sinh viên thực hiện

PHÂN CÔNG CÔNG VIỆC

Nhóm chia đồ án thành ba mảng có khối lượng tương đương, mỗi mảng gồm cả phần thiết kế thuật toán, phần cài đặt và phần kiểm thử tương ứng. Mỗi thành viên là người chịu trách nhiệm chính cho mảng của mình: viết mã, viết test, đo số liệu và soạn các mục báo cáo liên quan. Việc rà soát chéo mã nguồn được thực hiện cho toàn bộ ba mảng.

Bảng phân công nhiệm vụ giữa các thành viên

Thành viên	Nội dung phụ trách chính	Sản phẩm bàn giao
Hoàng Tiến Đạt 23010864	**Tầng tối ưu lộ trình.** Cài đặt Dijkstra nhiều đích trên MultiDiGraph, xây ma trận chi phí có hướng, quy hoạch động Held–Karp cho ba kiểu kết thúc, hai heuristic nearest-neighbor và 2-opt, thuật toán khôi phục tuyến chi tiết. Thiết kế và chạy benchmark tầng thứ tự ghé.	`src/shortest_path.py`, `src/tsp.py`, `src/heuristics.py`, `src/route_optimizer.py`, `scripts/benchmark_tsp.py`, `tests/test_shortest_path.py`, `tests/test_tsp.py`, `tests/test_heuristics.py`; Mục 2.2–2.3, 4.3–4.4, 5.4
Phạm Văn Tuấn Kiệt 23010881	**Mô hình truy tìm thông tin ẩn.** Thiết kế luật trò chơi và thứ tự ưu tiên kết thúc, kiểu `Observation` tách ranh giới thông tin, cơ chế lan truyền tập niềm tin, ba chiến thuật truy tìm và hàm điểm phối hợp. Thiết kế và chạy benchmark 360 ván.	`src/chase.py`, `src/chase_maps.py`, `scripts/benchmark_chase.py`, `scripts/prepare_chase_map.py`, `tests/test_chase.py`, `tests/test_chase_maps.py`; Mục 2.4–2.6, 3.5, 4.5–4.6, 5.5–5.7
Mầu Danh Chiến 23017270	**Dữ liệu, giao diện và chất lượng.** Chuẩn bị và đóng gói graph Hà Nội, quản lý tệp lớn bằng Git LFS, lớp nạp/cache và kiểm tra metadata, giao diện Streamlit hai chế độ, hiển thị bản đồ Folium, replay và xuất log JSON. Kiểm thử tích hợp, kiểm tra trình duyệt thật, biên soạn báo cáo LaTeX và sinh hình vẽ.	`app.py`, `src/map_service.py`, `src/chase_ui.py`, `src/visualization.py`, `scripts/preload_hanoi*.py`, `scripts/make_report_figures.py`, `tests/test_app*.py`, `tests/test_services.py`; Chương 1, 3, 6, 7 và phụ lục

Thành viên	Nội dung phụ trách chính	Sản phẩm bàn giao
Cả nhóm	Thống nhất mô hình toán và giao diện giữa các mô-đun, rà soát chéo mã nguồn, chạy lại toàn bộ 112 ca kiểm thử trước mỗi mốc bàn giao, dựng kịch bản demo và chuẩn bị nội dung bảo vệ.	Tóm tắt, Abstract, kịch bản demo, bộ số liệu trong `output/`

Khối lượng được cân bằng theo ba tiêu chí: số mô-đun phụ trách, số ca kiểm thử phải viết và số mục báo cáo phải soạn. Hai mảng thuật toán (Đạt và Kiệt) nặng về chứng minh tính đúng và phân tích độ phức tạp; mảng còn lại (Chiến) nặng về xử lý dữ liệu thật, tích hợp và bảo đảm chất lượng. Ba mảng đều có phần đo đạc riêng và đều cần thiết để sản phẩm chạy được đầu–cuối.

TÓM TẮT

Đồ án xây dựng một ứng dụng thuật toán trên mạng đường Hà Nội gồm hai chế độ có chung nền tảng đồ thị. Chế độ thứ nhất giải bài toán lập lộ trình qua nhiều địa điểm: Dijkstra được dùng để tính đường đi ngắn nhất giữa các điểm quan tâm, Held–Karp tìm thứ tự ghé tối ưu chính xác cho tối đa 10 điểm, còn láng giềng gần nhất và 2-opt đóng vai trò phương pháp so sánh. Hệ thống hỗ trợ ba kiểu kết thúc tuyến và hai hàm mục tiêu: tổng quãng đường hoặc tổng thời gian ước tính.

Chế độ thứ hai mô phỏng trò chơi truy tìm theo lượt trên đồ thị. Người trốn biết vị trí thật của mình, trong khi máy chỉ được quan sát tập vị trí khả dĩ, dấu vết tiết lộ theo chu kỳ, vị trí các đội truy tìm và các lối thoát. Ba chiến thuật *đuổi dấu vết*, *tìm kiếm độc lập* và *phối hợp chặn lối thoát* được xây dựng trên khoảng cách Dijkstra. Ranh giới thông tin được tách rõ trong kiểu dữ liệu quan sát nhằm ngăn thuật toán máy đọc trực tiếp trạng thái ẩn.

Phiên bản được kiểm tra ngày 24/10/2026 vượt qua 112/112 ca kiểm thử. Graph định tuyến đóng gói chứa 350.930 nút và 864.982 cạnh. Smoke test năm địa điểm cho kết quả 26,276 km khi tối ưu quãng đường và 29,93 phút khi tối ưu thời gian. Benchmark riêng cho tầng thứ tự ghé trên 24 cấu hình cho thấy Held–Karp chỉ tốn dưới 7,2 ms ở giới hạn 10 điểm, trong khi láng giềng gần nhất lệch tới 19,53% và 2-opt lệch tới 3,97% so với nghiệm tối ưu. Benchmark trò chơi gồm 360 ván cho thấy hiệu quả chiến thuật phụ thuộc mạnh vào kiểu di chuyển của người trốn và cấu trúc sân; do đó báo cáo không tuyên bố một heuristic luôn vượt trội. Kết quả minh họa giá trị của việc kết hợp thuật toán chính xác, heuristic và kiểm thử bất biến trong một ứng dụng đồ thị thực tế.

Từ khóa: đồ thị đường phố, Dijkstra, Held–Karp, bài toán người du lịch, pursuit–evasion, tập niềm tin, OpenStreetMap, Streamlit.

ABSTRACT

This project presents a graph-algorithm application with two integrated modes on the Hanoi road network. The route optimizer uses Dijkstra's algorithm to construct a pairwise cost matrix and Held–Karp dynamic programming to obtain an exact visiting order for at most ten destinations. Nearest-neighbor and 2-opt heuristics provide practical baselines. The application supports open routes, fixed final destinations, closed tours, and either distance or estimated travel time as the objective.

The second mode is a turn-based pursuit–evasion simulation with imperfect information. The hidden runner state is separated from the observation provided to the hunter policies. A belief set is propagated through graph reachability and periodically collapsed when the runner is revealed. Three hunter policies are compared: trail following, independent coverage search, and joint exit-aware containment.

On 24 October 2026, all 112 automated tests passed. The bundled routing graph contains 350,930 nodes and 864,982 edges. A five-location offline smoke test produced a 26.276 km distance-optimized route and a 29.93 minute time-optimized route. A dedicated benchmark over 24 configurations (4 to 11 points, three termination modes) shows Held–Karp staying under 7.2 ms at the supported limit while nearest-neighbor deviates from the optimum by up to 19.53 per cent and 2-opt by up to 3.97 per cent, which quantifies the price of dropping the optimality guarantee. A 360-game benchmark demonstrates that policy effectiveness depends on the runner behavior and arena topology; the results do not establish universal optimality of any heuristic.

Keywords: street graph, Dijkstra, Held–Karp, traveling salesperson problem, pursuit–evasion, belief state, OpenStreetMap, Streamlit.

MỤC LỤC

Lời Cảm Ơn

Lời Cam Đoan

Phân Công Công Việc

Tóm Tắt

Abstract

DANH MỤC HÌNH ẢNH

DANH MỤC BẢNG

DANH MỤC TỪ VIẾT TẮT

1 Giới Thiệu **1**

1.1 Bối cảnh 1

1.2 Bài toán thực tế 1

1.3 Câu hỏi và mục tiêu 2

1.4 Phạm vi và giả định 3

1.5 Đóng góp của đồ án 3

2 Cơ Sở Lý Thuyết **4**

2.1 Biểu diễn mạng đường bằng đồ thị 4

2.2 Thuật toán Dijkstra 4

2.3 Bài toán thứ tự ghé nhiều điểm 5

2.3.1 Quy hoạch động Held–Karp 5
2.3.2 Láng giềng gần nhất và 2-opt 6
2.4 Pursuit–evasion trên đồ thị 6
2.5 Ba chiến thuật truy tìm 7
2.6 Bất biến và thứ tự ưu tiên kết thúc 8

3 Phân Tích Và Thiết Kế Hệ Thống 9
3.1 Yêu cầu chức năng 9
3.2 Yêu cầu phi chức năng 10
3.3 Kiến trúc tổng thể 10
3.4 Luồng tối ưu lộ trình 11
3.5 Thiết kế trạng thái trò chơi 11
3.6 Thiết kế dữ liệu 12
3.7 Mô hình lỗi và xử lý 13

4 Cài Đặt Hệ Thống 14
4.1 Công nghệ sử dụng 14
4.2 Cấu trúc mã nguồn 15
4.3 Cài đặt Dijkstra 15
4.4 Cài đặt Held–Karp và khôi phục tuyến 16
4.5 Cài đặt Arena và tiền tính khoảng cách 16
4.6 Cài đặt một lượt chơi 17
4.7 Tích hợp giao diện và session state 17
4.8 Chuẩn bị và bảo toàn dữ liệu 17
4.9 Trích đoạn mã nguồn cốt lõi 18
4.9.1 Vòng lặp quy hoạch động Held–Karp 18
4.9.2 Lan truyền tập niềm tin trong một lượt 19

4.9.3 Hàm điểm phối hợp và danh sách rút gọn 19

5 Thực Nghiệm Và Đánh Giá **21**

5.1 Mục tiêu và nguyên tắc đánh giá 21

5.2 Chiến lược kiểm thử 22

5.2.1 Phân bố 112 ca kiểm thử 22

5.2.2 Các nhóm trường hợp biên 23

5.3 Smoke test định tuyến trên graph thật 23

5.4 So sánh thuật toán chính xác và heuristic trên tầng thứ tự ghé 24

5.4.1 Thiết kế phép đo 24

5.4.2 Chất lượng nghiệm 24

5.4.3 Thời gian chạy và kiểm chứng độ phức tạp 27

5.4.4 Diễn giải và giới hạn của phép đo 28

5.5 Thiết kế benchmark trò chơi 29

5.6 Kết quả trên sân lưới mô phỏng 29

5.7 Kết quả trên sân OSM cục bộ 30

5.8 Đánh giá giao diện và tích hợp 31

5.9 Các mối đe dọa tới tính hợp lệ 32

6 Minh Chứng Sản Phẩm Và Kịch Bản Demo **34**

6.1 Khởi chạy ứng dụng 34

6.2 Kịch bản demo chế độ Lộ trình 34

6.3 Kịch bản demo chế độ Trốn tìm 35

6.4 Thông điệp thuật toán cần nhấn mạnh 37

6.5 Tình huống lỗi và phương án phục hồi 37

7 Kết Luận Và Hướng Phát Triển **38**

7.1 Kết quả đạt được 38

7.2 Bài học thuật toán 38
7.3 Hạn chế 39
7.4 Hướng phát triển có thứ tự ưu tiên 39
7.5 Kết luận chung 40

A Phụ Lục Tái Lập **41**

A.1 Cài đặt trên Windows 11 41
A.2 Lệnh kiểm thử và benchmark 41
A.3 Cấu trúc mã nguồn và quy mô 41
A.4 Kết quả chạy bộ kiểm thử 43
A.5 Lược đồ tệp kết quả benchmark 44
A.6 Sinh lại hình vẽ của báo cáo 45
A.7 Ma trận trường hợp kết thúc trò chơi 46
A.8 Định nghĩa ký hiệu 46

TÀI LIỆU THAM KHẢO **47**

DANH SÁCH HÌNH VẼ

3.1 Kiến trúc hai chế độ dùng chung giao diện nhưng tách graph theo mục đích 10

3.2 Pipeline tối ưu lộ trình 11

3.3 Ranh giới thông tin giữa trạng thái ẩn và chính sách máy 12

5.1 Thời gian Held–Karp và khoảng cách chất lượng của heuristic 27

5.2 Số ván bắt được theo chiến thuật máy và bot người trốn 31

5.3 Tuyến thực được vẽ sau khi tối ưu trong giao diện tích hợp 32

6.1 Phòng điều khiển và trạng thái đầu ván 35

6.2 Giao diện trò chơi trên graph phố Hà Nội cục bộ 36

DANH SÁCH BẢNG

1.1 Phạm vi được thực hiện và nội dung không tuyên bố 3

3.1 Yêu cầu chức năng chính 9

3.2 Các nguồn dữ liệu của hệ thống 12

4.1 Một số thư viện chính trong phiên bản được kiểm tra 14

4.2 Trách nhiệm của các mô-đun chính 15

5.1 Số ca kiểm thử theo mô-đun 22

5.2 Kết quả smoke test năm địa điểm ngày 24/10/2026 23

5.3 Chi phí nghiệm theo mục tiêu quãng đường; gap so với Held–Karp 25

5.4 Thời gian trung bình của ba thuật toán theo số điểm (kiểu chu trình) . . . 27

5.5 Thời gian lập ma trận chi phí bằng Dijkstra theo số điểm 28

5.6 Benchmark sân lưới 81 nút; mỗi dòng gồm 20 ván 29

5.7 Benchmark sân OSM 358 nút; mỗi dòng gồm 20 ván 30

6.1 Thông điệp bảo vệ đồ án và bằng chứng tương ứng 37

A.1 Quy mô mã nguồn theo nhóm tệp 43

A.2 Thứ tự ưu tiên các điều kiện trong một lượt 46

A.3 Ký hiệu toán học sử dụng trong báo cáo 46

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Dạng đầy đủ	Giải thích ngắn
API	Application Programming Interface	Giao diện lập trình giữa các mô-đun
BFS	Breadth-First Search	Tìm kiếm theo chiều rộng
CSV	Comma-Separated Values	Định dạng bảng dữ liệu dạng văn bản
DP	Dynamic Programming	Quy hoạch động
GUI	Graphical User Interface	Giao diện đồ họa người dùng
JSON	JavaScript Object Notation	Định dạng dữ liệu dùng cho log và benchmark
LFS	Large File Storage	Cơ chế Git quản lý tệp dung lượng lớn
MCTS	Monte Carlo Tree Search	Tìm kiếm cây Monte Carlo
NN	Nearest Neighbor	Heuristic láng giềng gần nhất
OSM	OpenStreetMap	Nguồn dữ liệu bản đồ mở
PBF	Protocolbuffer Binary Format	Định dạng nén của dữ liệu OSM
POMDP	Partially Observable Markov Decision Process	Quá trình quyết định Markov quan sát một phần
TSP	Travelling Salesman Problem	Bài toán người du lịch
UI	User Interface	Giao diện người dùng

Từ viết tắt	Dạng đầy đủ	Giải thích ngắn
VRP	Vehicle Routing Problem	Bài toán định tuyến phương tiện
2-opt	Two-opt local search	Tìm kiếm cục bộ đảo đoạn trong tuyến

CHƯƠNG 1

GIỚI THIỆU

1.1 Bối cảnh

Đồ thị là một mô hình trung tâm của thuật toán ứng dụng. Trong giao thông, nút biểu diễn giao lộ hoặc điểm trên đường, cạnh biểu diễn đoạn đường và trọng số biểu diễn chi phí di chuyển. Cùng một cấu trúc đó còn mô tả được những bài toán động như tìm kiếm, truy đuổi và ra quyết định khi thông tin không đầy đủ. Đồ án khai thác điểm chung này để xây dựng một ứng dụng vừa có giá trị thực hành, vừa cho phép quan sát trực tiếp hành vi của nhiều lớp thuật toán.

Phần lõi của đề tài là tối ưu lộ trình đi qua nhiều địa điểm tại Hà Nội. Nếu chỉ cần đi từ một nguồn tới một đích, bài toán có thể giải bằng đường đi ngắn nhất; đây cũng là nền của các hệ thống định tuyến quy mô lớn [1]. Khi cần chọn cả thứ tự ghé nhiều điểm, không gian phương án tăng theo giai thừa; vì vậy hệ thống kết hợp Dijkstra với quy hoạch động Held–Karp. Phần mở rộng "Bắt tôi nếu có thể" biến mạng đường thành một sân pursuit–evasion theo lượt. Ở đây khó khăn không chỉ nằm ở tìm đường, mà còn ở việc ra quyết định dựa trên một tập vị trí khả dĩ thay vì biết chính xác đối thủ đang ở đâu.

1.2 Bài toán thực tế

Người dùng cần chọn một điểm xuất phát và từ một đến mười điểm cần ghé. Hệ thống phải trả về thứ tự ghé, tuyến đường chi tiết, tổng quãng đường và thời gian ước tính. Ba dạng kết thúc được hỗ trợ:

- **Tuyến mở**: điểm kết thúc do thuật toán tự chọn;
- **Điểm cuối cố định**: người dùng chỉ định điểm cuối;
- **Chu trình**: quay lại điểm xuất phát.

Trên cùng giao diện, người dùng có thể chuyển sang trò chơi truy tìm. Người trốn di chuyển trước với ngân sách tính theo mét. Hai đội máy di chuyển sau, chỉ dựa trên quan sát công khai. Người trốn thắng khi tới một lối thoát chưa bị chiếm; máy thắng nếu bắt

được người trốn hoặc người trốn hết số lượt. Vị trí thật của người trốn chỉ được tiết lộ theo chu kỳ cấu hình.

1.3 Câu hỏi và mục tiêu

Đồ án trả lời bốn câu hỏi kỹ thuật:

1. Làm thế nào chuyển mạng đường có hướng thành ma trận chi phí giữa các điểm quan tâm mà vẫn khôi phục được tuyến đường thật?
2. Làm thế nào tìm thứ tự ghé tối ưu chính xác trong giới hạn nhỏ, đồng thời có baseline heuristic để đối chiếu?
3. Làm thế nào mô hình hóa trạng thái ẩn của người trốn để chính sách máy không vô tình đọc thông tin bí mật?
4. Làm thế nào kiểm thử các bất biến thuật toán và quy tắc kết thúc thay vì chỉ kiểm thử giao diện?

Mục tiêu sản phẩm là một ứng dụng chạy cục bộ, không cần gọi dịch vụ định tuyến trong luồng tối ưu thông thường; cung cấp phản hồi trực quan trên bản đồ; và duy trì được trạng thái khi chuyển qua lại giữa hai chế độ. Mục tiêu học thuật là trình bày rõ mô hình toán, giả định, độ phức tạp, điều kiện đúng và giới hạn của từng thuật toán.

1.4 Phạm vi và giả định

Bảng 1.1: Phạm vi được thực hiện và nội dung không tuyên bố

Trong phạm vi	**Ngoài phạm vi hoặc chưa được chứng minh**
Graph ô tô Hà Nội đóng gói cục bộ; tối đa 10 điểm ghé	Dữ liệu giao thông thời gian thực, ùn tắc hoặc đóng đường phát sinh tức thời
Tối ưu chính xác thứ tự ghé theo ma trận chi phí tĩnh	Tối ưu toàn cục cho số điểm lớn tùy ý hoặc bài toán VRP nhiều xe [2]
Trò chơi theo lượt, bắt tại nút cuối bước, tối đa hai đội máy	Va chạm giữa cạnh, chuyển động liên tục hoặc cân bằng Nash
So sánh ba heuristic trên bot và seed cố định	Chứng minh chiến thuật máy tối ưu hoặc tổng quát hóa cho người chơi thật

Chi phí thời gian trong graph là ước lượng tĩnh được lưu trên cạnh. Vì vậy việc chọn mục tiêu thời gian không đồng nghĩa với định tuyến phụ thuộc thời gian. Tương tự, graph trò chơi OSM đi kèm được trích từ một nguồn tải dở và điều chỉnh hai chiều để phục vụ mô phỏng; nó không được dùng làm hướng dẫn giao thông.

1.5 Đóng góp của đồ án

Các kết quả chính của đồ án gồm:

- Một pipeline hoàn chỉnh từ điểm địa lý tới đường đi chi tiết: ánh xạ nút, Dijkstra nhiều đích, ma trận chi phí, Held–Karp và khôi phục tuyến.
- Một giao diện hợp nhất hai chế độ, giữ độc lập trạng thái lộ trình và ván chơi khi người dùng chuyển màn hình.
- Một mô hình pursuit–evasion thông tin ẩn với kiểu `Observation` không chứa vị trí hoặc lịch sử bí mật của người trốn.
- Ba chính sách truy tìm có giải thích, benchmark 360 ván và bộ 112 kiểm thử bao phủ thuật toán, dữ liệu, quy tắc biên và tích hợp giao diện.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Biểu diễn mạng đường bằng đồ thị

Mạng đường được mô hình hóa bởi đa đồ thị có hướng có trọng số $G = (V, E, w)$. Mỗi $v \in V$ có tọa độ (x_v, y_v). Một cạnh $e = (u, v, k) \in E$ là một đoạn đường từ u tới v; chỉ số k cho phép tồn tại nhiều cạnh song song. Trọng số $w(e) > 0$ có thể là chiều dài theo mét hoặc thời gian ước tính theo giây. Việc dùng graph có hướng giữ được đường một chiều và sự khác nhau giữa hai hướng di chuyển.

Chi phí của một đường đi

Với đường đi $P = (v_0, e_1, v_1, \ldots, e_m, v_m)$, chi phí là

$$C(P) = \sum_{i=1}^{m} w(e_i).$$

Bài toán đường đi ngắn nhất tìm $P^* \in \mathcal{P}(s,t)$ sao cho

$$P^* = \arg\min_{P \in \mathcal{P}(s,t)} C(P).$$

Dữ liệu đường được lấy từ OpenStreetMap và xử lý bằng OSMnx [3, 4]. Graph định tuyến đóng gói hiện tại có 350.930 nút, 864.982 cạnh và hộp bao địa lý $[105{,}2; 20{,}5; 106{,}1; 21{,}45]$ theo thứ tự kinh độ tây, vĩ độ nam, kinh độ đông, vĩ độ bắc.

2.2 Thuật toán Dijkstra

Dijkstra giải bài toán đường đi ngắn nhất nguồn đơn khi mọi trọng số cạnh không âm [5, 6]. Thuật toán duy trì khoảng cách tạm thời $d[v]$, nút trước $\pi[v]$ và một hàng đợi ưu tiên. Khi nút có khoảng cách nhỏ nhất được lấy ra và khoảng cách đó chưa lỗi thời, giá trị của nó đã được chốt.

Phép nới lỏng cạnh

Với cạnh (u, v), nếu

$$d[u] + w(u, v) < d[v],$$

thì cập nhật $d[v] \leftarrow d[u] + w(u, v)$ và $\pi[v] \leftarrow u$.

Trong đồ án, Dijkstra nhận một tập đích và có thể dừng khi tất cả các đích đã được chốt. Với heap nhị phân, độ phức tạp xấp xỉ $O((|V|+|E|)\log|V|)$ cho mỗi nguồn; bộ nhớ $O(|V|)$. Hàng đợi Fibonacci hạ cận lý thuyết xuống $O(|E| + |V|\log|V|)$ nhưng hằng số lớn hơn nên không được dùng ở đây [7]. Để lập lộ trình qua n điểm quan tâm, thuật toán được chạy từ từng điểm, tạo ma trận $D \in \mathbb{R}^{n\times n}$, trong đó D_{ij} là chi phí từ điểm i tới điểm j. Do graph có hướng, nói chung $D_{ij} \neq D_{ji}$.

Điều kiện quan trọng là $w(e) > 0$ và các điểm cần thiết phải nối được theo hướng di chuyển. Nếu không có đường, giá trị được biểu diễn bằng $+\infty$ và tầng tối ưu thứ tự phải báo lỗi thay vì tạo một tuyến không hợp lệ.

2.3 Bài toán thứ tự ghé nhiều điểm

Sau khi có ma trận D, bài toán trở thành một biến thể TSP bất đối xứng. Nút 0 là điểm xuất phát; các nút còn lại là điểm cần ghé. Tùy chế độ, tuyến có thể kết thúc tự do, kết thúc tại một điểm cố định hoặc quay lại 0.

2.3.1 Quy hoạch động Held–Karp

Held–Karp dùng trạng thái (S, j), trong đó S là tập điểm đã ghé và $j \in S$ là điểm cuối hiện tại [8]. Gọi $F(S, j)$ là chi phí nhỏ nhất để đi từ 0, ghé đúng các điểm trong S và kết thúc tại j.

Truy hồi Held–Karp

$$F(\{j\}, j) = D_{0j},$$

$$F(S, j) = \min_{i\in S\setminus\{j\}} \left[F(S \setminus \{j\}, i) + D_{ij}\right].$$

Với tuyến mở, chọn $\min_j F(U, j)$. Với chu trình, chọn $\min_j[F(U, j)+D_{j0}]$. Điểm cuối cố định t dùng trực tiếp $F(U, t)$.

Thuật toán có độ phức tạp thời gian $O(n^2 2^n)$ và bộ nhớ $O(n2^n)$ [6, 9]. Giới hạn 10 điểm ghé của giao diện giữ số trạng thái ở mức phù hợp cho tương tác. Mảng nút trước được lưu cùng chi phí để khôi phục thứ tự tối ưu, sau đó từng cặp liên tiếp được thay bằng đường đi Dijkstra tương ứng.

2.3.2 Láng giềng gần nhất và 2-opt

Láng giềng gần nhất chọn điểm chưa ghé có chi phí trực tiếp nhỏ nhất [10]. Chi phí tính toán là $O(n^2)$ nhưng kết quả phụ thuộc mạnh vào lựa chọn cục bộ. 2-opt [11] bắt đầu từ một tuyến và đảo thứ tự một đoạn nếu phép đổi làm giảm tổng chi phí. Với ma trận bất đối xứng, hệ thống đánh giá lại toàn bộ chi phí ứng viên thay vì sử dụng công thức delta dành riêng cho TSP đối xứng.

Khoảng cách thực đo của hai heuristic so với nghiệm chính xác được trình bày ở mục 5.4.

Hai heuristic được dùng làm baseline và phương án so sánh, không thay thế Held–Karp trong miền tối đa 10 điểm. Không có bảo đảm chúng tìm được tối ưu toàn cục.

2.4 Pursuit–evasion trên đồ thị

Pursuit–evasion trên graph là một họ bài toán trong đó bên truy tìm và bên lẩn trốn di chuyển trên các nút/cạnh theo luật xác định [12, 13]. Biến thể của đồ án là rời rạc theo lượt, có lối thoát, ngân sách khoảng cách và quan sát không đầy đủ.

Trạng thái đầy đủ ở lượt t gồm vị trí thật r_t của người trốn, vị trí các đội $H_t = (h_t^1, h_t^2)$, tập lối thoát X, tập niềm tin B_t và lịch sử cần thiết. Chính sách máy không nhận toàn bộ trạng thái; đầu vào của nó là quan sát $O_t = (t, H_t, X, B_t, \ell, \tau, \rho)$, trong đó ℓ là vị trí được thấy gần nhất, τ là lượt thấy gần nhất và ρ là bộ luật.

Lan truyền tập niềm tin

Với ngân sách người trốn b_r và khoảng cách ngắn nhất d, tập niềm tin trước khi áp dụng thông tin mới là

$$\widehat{B}_{t+1} = \{v \in V \mid \exists u \in B_t : \ d(u, v) \leq b_r\}.$$

Nếu lượt $t+1$ là lượt tiết lộ thì $B_{t+1} = \{r_{t+1}\}$. Nếu không, các nút đang bị đội truy tìm chiếm và đã được loại trừ bởi sự sống sót của người trốn sẽ được bỏ khỏi $\widehat{B}_{t+1}$.

Tập niềm tin là biểu diễn chính xác theo luật chuyển trạng thái đã cài đặt, nhưng không phải phân phối xác suất. Mọi vị trí khả dĩ được coi như nhau; vì vậy chính sách coverage là heuristic dựa trên khoảng cách, không phải suy luận Bayes. Dạng tổng quát của bài toán ra quyết định dưới quan sát một phần là POMDP, trong đó trạng thái niềm tin là một phân phối xác suất chứ không phải một tập [14].

2.5 Ba chiến thuật truy tìm

Gọi b_h là ngân sách mỗi đội. Điểm bao phủ của vị trí ứng viên q đối với tập S được định nghĩa:

Điểm bao phủ nghịch đảo khoảng cách

$$\mathrm{cov}(q, S) = \frac{1}{|S|} \sum_{p \in S} \frac{1}{1 + d(q,p)/b_h}.$$

Đuổi dấu vết đưa mỗi đội theo đường ngắn nhất về vị trí được thấy gần nhất. **Tìm kiếm độc lập** để mỗi đội chọn riêng vị trí khả đạt có điểm bao phủ lớn nhất. **Phối hợp chặn lối thoát** xét tích Descartes của tối đa tám ứng viên tốt nhất cho mỗi đội, bổ sung các bước hướng lối thoát, rồi tối đa hóa:

Điểm phối hợp

$$J(M) = \frac{1}{|B|} \sum_{p \in B} \max_{m \in M} \frac{1}{1 + d(m,p)/b_h} + 0{,}35 \cdot \frac{1}{|X|} \sum_{x \in X} \max_{m \in M} \frac{1}{1 + d(m,x)/b_h}.$$

Hệ số 0,35 là tham số thiết kế của phiên bản hiện tại, chưa được tối ưu bằng một tập validation độc lập. Việc cắt còn tám ứng viên giữ thời gian phản hồi nhỏ, nhưng cũng có thể loại một nước đi tốt; đây là một giới hạn có chủ ý của heuristic.

2.6 Bất biến và thứ tự ưu tiên kết thúc

Các bất biến được dùng để kiểm thử gồm: người trốn luôn thuộc tập niềm tin khi ván đang diễn ra; mọi bước đi nằm trong ngân sách và đúng chiều cạnh; chính sách máy không nhận vị trí ẩn; tập niềm tin không rỗng trong trạng thái hợp lệ; và trạng thái kết thúc không được đi tiếp.

CHƯƠNG 3

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu chức năng

Bảng 3.1: Yêu cầu chức năng chính

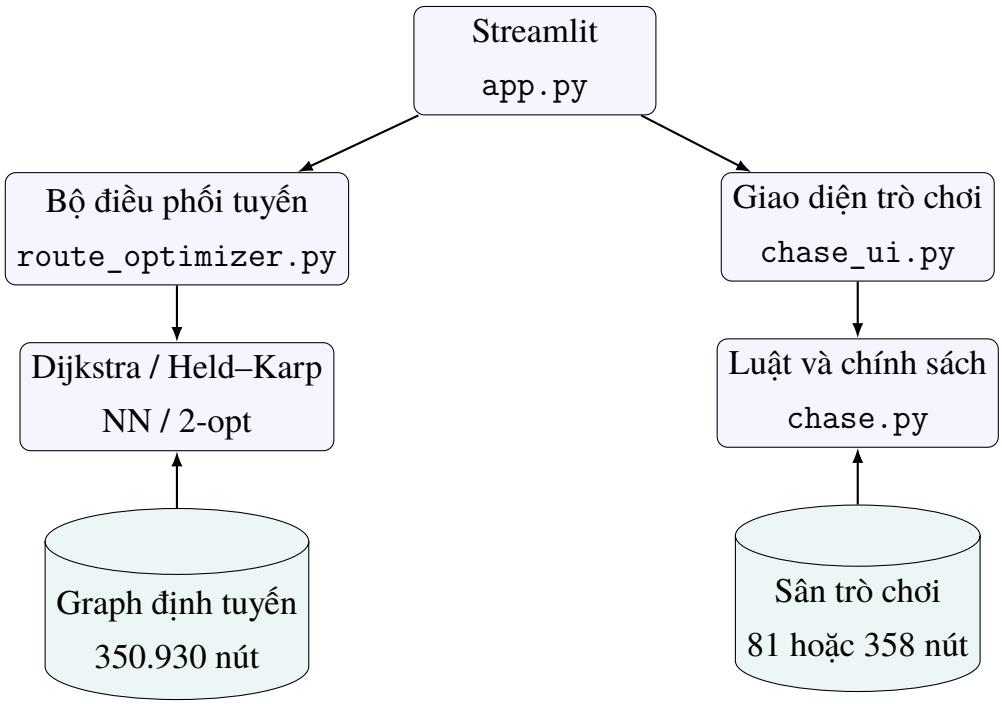
Mã	Chức năng	Mô tả
F01	Chọn địa điểm	Chọn từ danh sách, tìm tên/địa chỉ hoặc nhấp bản đồ; điểm được ánh xạ về nút graph gần nhất.
F02	Thiết lập mục tiêu	Tối ưu tổng chiều dài hoặc tổng thời gian ước tính.
F03	Thiết lập điểm cuối	Tuyến mở, điểm cuối cố định hoặc quay về điểm đầu.
F04	Tối ưu và hiển thị	Hiển thị thứ tự ghé, polyline, khoảng cách, thời gian và thời gian chạy thuật toán.
F05	Chuyển chế độ	Chuyển giữa Lộ trình và Trốn tìm mà không mất trạng thái của chế độ còn lại.
F06	Cấu hình ván	Chọn sân, chiến thuật, seed, ngân sách, chu kỳ tiết lộ và giới hạn lượt.
F07	Chơi theo lượt	Chỉ cho chọn nút khả đạt; cập nhật tập niềm tin, nước máy và trạng thái thắng/thua.
F08	Phát lại và xuất log	Xem lại từng lượt và tải lịch sử ván ở dạng JSON.

3.2 Yêu cầu phi chức năng

Hệ thống phải chạy cục bộ trên Python 3.11; graph định tuyến được tải một lần và cache trong phiên; lỗi dữ liệu phải hiện thành thông báo có thể hiểu thay vì làm sập giao diện. Phần trò chơi phải chạy ngay cả khi graph định tuyến lớn không có sẵn. Với tối đa 10 điểm ghé, thời gian Held–Karp cần nhỏ so với bước tính ma trận đường đi. Các trạng thái và cấu hình phải có thể tái lập bằng seed.

Về độ tin cậy, dữ liệu biên phải được kiểm tra: tọa độ hữu hạn, trọng số dương, ngân sách dương, graph trò chơi từ 3 đến 600 nút, điểm xuất phát/đội/lối thoát không trùng nhau và người trốn có thể tới ít nhất một lối thoát. Đây là các nghĩa vụ triển khai tương ứng với giả định trong mô hình toán.

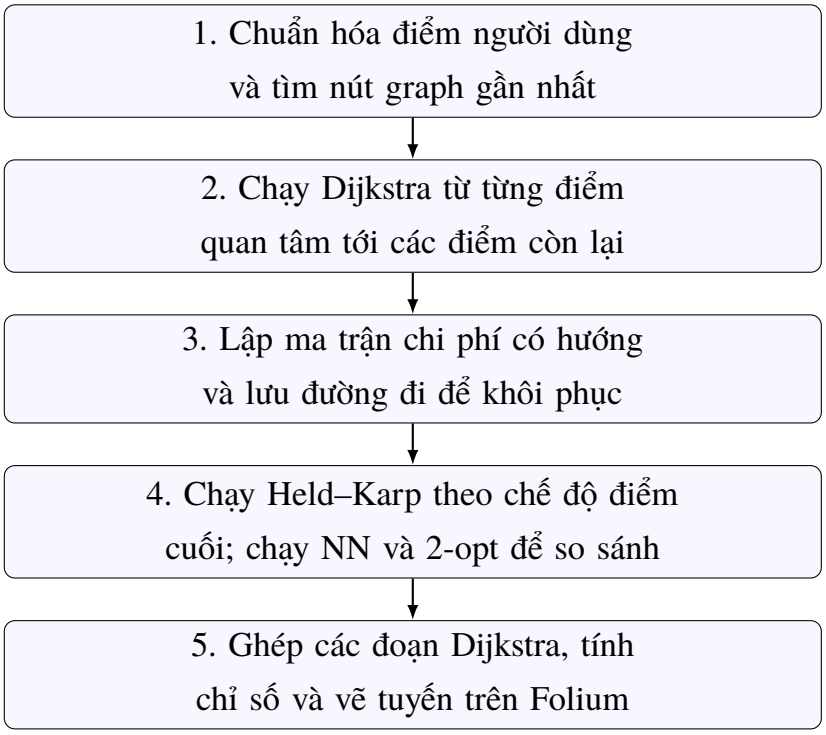
3.3 Kiến trúc tổng thể



Hình 3.1: Kiến trúc hai chế độ dùng chung giao diện nhưng tách graph theo mục đích

Việc tách hai graph giải quyết hai nhu cầu khác nhau. Định tuyến cần mạng lớn và giữ đúng chiều đường ô tô. Trò chơi cần graph nhỏ để tiền tính khoảng cách mọi cặp và phản hồi nhanh mỗi lượt. Không dùng graph trò chơi để thay thế graph định tuyến khi tệp lớn thiếu, vì việc đó sẽ làm thay đổi ngầm ý nghĩa ứng dụng.

3.4 Luồng tối ưu lộ trình



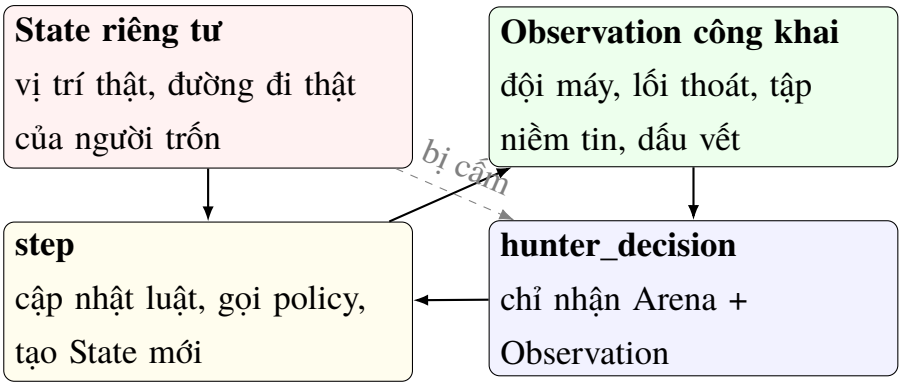
Hình 3.2: Pipeline tối ưu lộ trình

Điểm tách quan trọng là Dijkstra tối ưu từng đoạn trên graph đường, còn Held–Karp tối ưu thứ tự trên graph đầy đủ của các điểm quan tâm. Do chi phí của mỗi cung trong graph đầy đủ đã là đường đi ngắn nhất, ghép các cung theo thứ tự Held–Karp tạo tuyến tối ưu đối với mô hình chi phí tĩnh và tập điểm đã ánh xạ.

3.5 Thiết kế trạng thái trò chơi

Ba kiểu dữ liệu bất biến được sử dụng:

- `Rules`: ngân sách người trốn, ngân sách đội máy, chu kỳ tiết lộ, giới hạn lượt;
- `Observation`: chỉ chứa thông tin chính sách máy được phép biết;
- `State`: chứa vị trí thật của người trốn, quan sát công khai, trạng thái kết thúc và các đường đi của lượt gần nhất.



Hình 3.3: Ranh giới thông tin giữa trạng thái ẩn và chính sách máy

Thiết kế này biến một yêu cầu khái niệm thành nghĩa vụ có thể kiểm thử: hai trạng thái có cùng `Observation` nhưng vị trí người trốn khác nhau phải tạo cùng đầu vào và cùng quyết định máy. Tuy nhiên, điều này chỉ chứng minh ranh giới tại API chính sách hiện tại; nếu mã tương lai thêm biến toàn cục hoặc truy cập session state bên ngoài API, bất biến phải được kiểm tra lại.

3.6 Thiết kế dữ liệu

Bảng 3.2: Các nguồn dữ liệu của hệ thống

Nguồn	Quy mô	Mục đích	Giới hạn
`hanoi_urban_drive`	350.930 nút, 864.982 cạnh	Định tuyến ô tô trên phạm vi Hà Nội mở rộng	Tĩnh; không phản ánh ùn tắc thời gian thực
Lưới mô phỏng	81 nút, 288 cạnh	Sân chuẩn, đối xứng, dễ tái lập	Không phải đường thực tế
`chase_hanoi`	358 nút, 1.184 cạnh	Hình học phố Hà Nội cho trò chơi	Trích từ nguồn gián đoạn, điều chỉnh hai chiều

Cạnh dài trong sân trò chơi được chia thành các đoạn không quá 90 m. Phép chia bảo toàn tổng chiều dài và chiều cạnh; các cạnh ngược chiều dùng chung nút trung gian

khi tọa độ trùng. Mục đích là tạo đủ điểm dừng cho ngân sách 160 m/lượt, không phải làm tăng độ chính xác địa lý.

3.7 Mô hình lỗi và xử lý

Lỗi được chia thành ba lớp. Lỗi đầu vào gồm tọa độ sai, ngân sách không dương và điểm trùng; hệ thống từ chối sớm. Lỗi dữ liệu gồm tệp GraphML/Pickle thiếu hoặc không hợp lệ; giao diện vẫn mở và hiển thị cảnh báo, còn chế độ trò chơi độc lập vẫn dùng được. Lỗi kết nối chỉ ảnh hưởng geocoding, tải tile hoặc làm mới dữ liệu; thuật toán định tuyến trên graph đóng gói không phụ thuộc tile nền.

Phân biệt bản đồ nền và thuật toán

Tile bản đồ chỉ là ảnh nền trực quan. Một lỗi DNS hoặc chính sách tile có thể làm nền màu xám trong khi graph, Dijkstra và tuyến kết quả vẫn hoạt động. Vì vậy kiểm thử đúng phải kiểm tra cả giá trị thuật toán lẫn lớp hiển thị, không suy ra lỗi thuật toán chỉ từ việc tile không tải.

CHƯƠNG 4

CÀI ĐẶT HỆ THỐNG

4.1 Công nghệ sử dụng

Ứng dụng được viết bằng Python 3.11. Streamlit cung cấp giao diện và session state; NetworkX biểu diễn graph; OSMnx đọc/ghi dữ liệu OpenStreetMap; Folium và streamlit-folium hiển thị bản đồ; Shapely xử lý hình học cạnh; scikit-learn được OSMnx dùng trong tìm nút gần nhất [15, 16]. Các phiên bản trực tiếp được khóa trong `requirements.txt` để hạn chế sai khác môi trường.

Bảng 4.1: Một số thư viện chính trong phiên bản được kiểm tra

Thư viện	**Phiên bản**	**Vai trò**
Streamlit	1.50.0	Giao diện web cục bộ và quản lý phiên
OSMnx	2.0.6	Nạp graph OSM, ánh xạ tọa độ, tiện ích địa lý
NetworkX	3.5	Cấu trúc MultiDiGraph và kiểm chứng oracle trong test
Folium / streamlit-folium	0.20.0 / 0.25.3	Bản đồ, marker, sự kiện click
Shapely	2.2.0	Chia hình học cạnh dài
Pandas / NumPy	2.3.2 / 2.2.6	Bảng dữ liệu và tính toán hỗ trợ

4.2 Cấu trúc mã nguồn

Bảng 4.2: Trách nhiệm của các mô-đun chính

Mô-đun	Trách nhiệm
`app.py`	Điểm vào duy nhất, chuyển chế độ, widget và trình bày kết quả
`src/shortest_path.py`	Dijkstra tự cài đặt, nhiều đích, hỗ trợ MultiDiGraph
`src/tsp.py`	Held–Karp cho các chế độ kết thúc
`src/heuristics.py`	Nearest-neighbor và 2-opt
`src/route_optimizer.py`	Lập ma trận, gọi thuật toán và ghép tuyến
`src/map_service.py`	Nạp/cache graph, kiểm tra metadata và truy vấn mạng phụ
`src/chase.py`	Luật, trạng thái, Arena, tập niềm tin và chính sách máy
`src/chase_maps.py`	Sân lưới, sân OSM, chia cạnh và sinh kịch bản bằng seed
`src/chase_ui.py`	Điều khiển ván, bản đồ, replay và xuất JSON
`tests/`	Unit test, oracle test, AppTest và test dữ liệu

4.3 Cài đặt Dijkstra

Phiên bản trong dự án không gọi trực tiếp hàm shortest-path của NetworkX. Heap chứa bộ ba khoảng cách, thứ tự chèn và nút để tránh yêu cầu so sánh trực tiếp hai kiểu định danh nút. Với đa cạnh (u, v), thuật toán chọn trọng số nhỏ nhất hợp lệ giữa các cạnh song song. Đầu vào bị từ chối nếu trọng số là boolean, không phải số hữu hạn hoặc âm.

Test dùng NetworkX làm oracle trên nhiều graph sinh ngẫu nhiên. Oracle không làm cho cài đặt đúng một cách tự động, nhưng giúp phát hiện sai khác về khoảng cách trên nhiều cấu hình hơn các ví dụ viết tay.

Listing 4.1: Giả mã Dijkstra nhiều đích

```
distance[source] = 0
push(heap, (0, source))
while heap is not empty and targets remain:
    du, u = pop_min(heap)
    if du is stale: continue
    if u is a target: mark u settled
    for each outgoing neighbor v of u:
        weight = minimum valid parallel-edge weight(u, v)
        if du + weight < distance[v]:
            distance[v] = du + weight
            predecessor[v] = u
            push(heap, (distance[v], v))
return distances and reconstructed paths
```

4.4 Cài đặt Held–Karp và khôi phục tuyến

Tập điểm được mã hóa bằng bitmask. Khóa bảng động là $(mask, last)$; giá trị lưu chi phí và nút trước. Cấu hình điểm cuối được xử lý ở bước chọn trạng thái cuối, không nhân bản ba thuật toán. Trước khi chạy, ma trận được kiểm tra vuông, hữu hạn tại các cung cần thiết và nhất quán kích thước.

Sau khi khôi phục thứ tự, bộ điều phối lấy đường đi chi tiết đã lưu từ từng lần Dijkstra. Khi ghép hai đoạn liên tiếp, nút đầu của đoạn sau bị bỏ để không lặp nút nối. Tổng quãng đường và thời gian được tính từ cạnh thực của tuyến ghép, không suy ra từ khoảng cách Euclid giữa marker.

4.5 Cài đặt Arena và tiền tính khoảng cách

`Arena` sao chép rồi đóng băng graph để tránh thay đổi trong ván. Vì sân giới hạn tối đa 600 nút, hệ thống chạy Dijkstra từ mọi nguồn và lưu cả khoảng cách lẫn đường đi. Nếu $N = |V|$, chi phí khởi tạo xấp xỉ $O(N(|E| + |V|) \log |V|)$; đổi lại, truy vấn khoảng cách trong mỗi lượt là $O(1)$ và việc lấy các nút trong ngân sách chỉ quét danh sách nút.

Đây là lựa chọn phù hợp cho sân nhỏ, nhưng không mở rộng trực tiếp sang graph 350 nghìn nút. Nếu dùng sân lớn, cần chuyển sang Dijkstra theo nhu cầu, landmark, multi-source search hoặc cache có giới hạn.

4.6 Cài đặt một lượt chơi

Hàm `step` là điểm chuyển trạng thái duy nhất. Trình tự cài đặt như sau:

1. kiểm tra ván còn hoạt động và điểm đến nằm trong ngân sách;
2. tăng lượt, lan truyền tập niềm tin và áp dụng tiết lộ định kỳ;
3. xét bắt tại vị trí đội máy, sau đó xét thoát;
4. loại vị trí đội máy khỏi tập niềm tin khi người trốn còn sống;
5. gọi chính sách chỉ với `Observation`;
6. di chuyển đồng thời các đội, xét bắt rồi xét hết lượt;
7. trả về một `State` mới thay vì sửa trạng thái cũ.

Tính bất biến giúp replay đơn giản: lịch sử chỉ là danh sách các snapshot. JSON được xuất với schema version 1, nguồn sân, seed, chiến thuật và toàn bộ trạng thái cần thiết để kiểm tra diễn biến.

4.7 Tích hợp giao diện và session state

Hai nút *Lộ trình/Trốn tìm* cập nhật `workspace_mode` và query parameter. Nhánh trò chơi trả về trước khi graph định tuyến lớn được nạp, nhờ đó chế độ này vẫn mở nhanh. Các khóa widget định tuyến được giữ sống trong session state khi widget tạm ẩn; khi quay lại, tiêu chí, kiểu điểm cuối, lựa chọn điểm và tùy chọn click bản đồ vẫn còn.

Click bản đồ trò chơi sử dụng đối tượng marker trả về bởi streamlit-folium thay vì click nền chung. Chỉ các chấm xanh trong phạm vi di chuyển được chấp nhận; dropdown cung cấp phương án truy cập thay thế và giúp kiểm thử ổn định.

4.8 Chuẩn bị và bảo toàn dữ liệu

Graph lớn được quản lý bằng Git LFS. Tệp Pickle giúp nạp nhanh; GraphML là phương án dự phòng. Metadata chứa bbox, quy mô, nguồn và thời điểm tạo. Script PBF đọc dữ liệu Việt Nam cục bộ, cắt theo bbox, kiểm tra kết nối các hành lang quan trọng rồi mới thay gói hiện tại. Thiết kế này tránh phụ thuộc Overpass trong mỗi lần người dùng tối ưu.

Script chuẩn bị sân trò chơi dùng tệp tạm và chỉ thay tệp đích sau khi graph mới vượt kiểm tra. Test mô phỏng lỗi tải để bảo đảm tệp sân đang hoạt động không bị ghi đè bởi dữ liệu dở dang.

4.9 Trích đoạn mã nguồn cốt lõi

Ba đoạn dưới đây là mã thật trong dự án, không phải giả mã. Chúng được chọn vì mỗi đoạn tương ứng với một phát biểu lý thuyết ở Chương 2 và là chỗ dễ sai nhất khi cài đặt.

4.9.1 Vòng lặp quy hoạch động Held–Karp

Đoạn sau là phần truy hồi và ba cách chọn trạng thái cuối trong `src/tsp.py`. Điểm đáng chú ý là ba kiểu kết thúc chỉ khác nhau ở bước chọn trạng thái cuối, còn bảng `dp` dùng chung; nhờ vậy không có ba bản sao của cùng một thuật toán.

Listing 4.2: Truy hồi Held–Karp và ba kiểu kết thúc

```
for mask in range(1, 1 << n):
    for j in range(n):
        if not mask & (1 << j) or not isfinite(dp[mask][j]):
            continue
        for k in range(n):
            if mask & (1 << k):
                continue
            next_mask = mask | (1 << k)
            candidate = dp[mask][j] + costs[destinations[j]][
               destinations[k]]
            # Transition: noi them mot diem chua ghe vao cuoi tuyen.
            if candidate < dp[next_mask][k]:
                dp[next_mask][k] = candidate
                parent[next_mask][k] = j

full_mask = (1 << n) - 1
if return_to_start:
    # Closed TSP: chon diem cuoi tam thoi roi cong canh quay ve start.
    last = min(range(n),
               key=lambda j: dp[full_mask][j] + costs[destinations[j
                  ]][start_index])
    optimal_cost = dp[full_mask][last] + costs[destinations[last]][
       start_index]
elif end_index is not None:
    # Fixed end: trang thai cuoi bat buoc nam tai end_index.
    last = destinations.index(end_index)
    optimal_cost = dp[full_mask][last]
```

```
else:
    # Free end: lay min tren moi diem cuoi, KHONG cong canh ve start.
    last = min(range(n), key=lambda j: dp[full_mask][j])
    optimal_cost = dp[full_mask][last]
```

Việc kiểm tra `isfinite(dp[mask][j])` trước khi mở rộng là bắt buộc trên graph có hướng: nếu một trạng thái đã không tới được, mọi trạng thái mở rộng từ nó cũng vô nghĩa, và phép cộng với $+\infty$ sẽ lan giá trị vô hạn vào bảng. Khi `optimal_cost` vẫn vô hạn ở cuối, hệ thống báo lỗi cho người dùng thay vì trả về một tuyến không hợp lệ.

4.9.2 Lan truyền tập niềm tin trong một lượt

Đoạn sau trong `src/chase.py` thực hiện đúng công thức lan truyền ở Chương 2 và hai lần thu hẹp tập niềm tin.

Listing 4.3: Cập nhật tập niềm tin theo lượt

```
turn = obs.turn + 1
belief = arena.expand(obs.belief, obs.rules.runner_budget)
revealed = turn % obs.rules.reveal_every == 0
if revealed:
    belief = frozenset({destination})
obs = replace(obs, turn=turn, belief=belief,
              last_seen=destination if revealed else obs.last_seen,
              last_seen_turn=turn if revealed else obs.last_seen_turn)
...
# Surviving the runner phase also rules out all currently occupied
   hunter nodes.
obs = replace(obs, belief=obs.belief.difference(obs.hunters))
```

Dòng cuối là một suy luận chứ không phải một thủ thuật: nếu người trốn còn sống sau pha di chuyển của mình, thì vị trí hiện tại của người trốn không thể là một nút đang có đội truy tìm. Đây là thông tin hợp lệ rút ra từ quan sát công khai, nên chính sách máy được phép dùng. Ngược lại, biến `state.runner` không bao giờ xuất hiện trong `Observation`.

4.9.3 Hàm điểm phối hợp và danh sách rút gọn

Listing 4.4: Chiến thuật phối hợp chặn lối thoát

```
def joint_score(moves):
```

```
    belief_score = sum(max(1 / (1 + arena.distance(m, p) / budget) for
        m in moves)
                       for p in belief) / len(belief)
    exit_score = sum(max(1 / (1 + arena.distance(m, e) / budget) for m
        in moves)
                     for e in observation.exits) / len(observation.
                        exits)
    return belief_score + 0.35 * exit_score

# Include exit-directed moves so the coverage shortlist cannot hide a
    blockade.
for index, hunter in enumerate(hunters):
    pools[index] = list(dict.fromkeys([
        *pools[index], *(arena.advance(hunter, e, budget) for e in
            observation.exits)
    ]))
moves = max(product(*pools), key=joint_score)
```

Hai chi tiết triển khai đáng nói. Thứ nhất, `max` bên trong tổng chứ không phải tổng của mọi đội: một vị trí nghi ngờ chỉ cần *một* đội ở gần là đã được bao phủ, nên cộng dồn khoảng cách của cả hai đội sẽ thưởng cho việc hai đội đứng chồng lên nhau. Thứ hai, các nước đi hướng lối thoát được thêm vào danh sách rút gọn sau khi đã cắt còn tám ứng viên theo coverage; nếu không, một nước chặn lối thoát tốt có thể bị loại ngay ở bước cắt và hệ số 0,35 sẽ không bao giờ có cơ hội phát huy tác dụng.

CHƯƠNG 5

THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Mục tiêu và nguyên tắc đánh giá

Thực nghiệm nhằm kiểm tra ba lớp khác nhau: (1) tính đúng của thuật toán và luật chuyển trạng thái; (2) khả năng chạy trên graph đóng gói thật; (3) hành vi tương đối của các heuristic truy tìm. Một ca test đạt không tự động chứng minh hiệu quả trên người dùng thật; một benchmark bot cũng không chứng minh chính sách tối ưu. Các kết luận dưới đây được giới hạn theo đúng bằng chứng của từng phép đo.

Các kết quả trong chương được chạy lại ngày 24/10/2026 trên máy phát triển dùng Python 3.11. Thời gian là thời gian thực đo của một lần chạy, chịu ảnh hưởng tải CPU, bộ nhớ và cache; vì vậy chỉ dùng để mô tả cấu hình hiện tại.

5.2 Chiến lược kiểm thử

5.2.1 Phân bố 112 ca kiểm thử

Bảng 5.1: Số ca kiểm thử theo mô-đun

Tệp	Số test	Trọng tâm
`test_chase.py`	38	Luật, tập niềm tin, ba chiến thuật, terminal priority
`test_services.py`	17	Nạp graph, bbox, lỗi mạng/dữ liệu, cache
`test_chase_maps.py`	11	Sân, chia cạnh, seed, GraphML gián đoạn
`test_tsp.py`	11	Held–Karp, ba kiểu kết thúc, ma trận bất đối xứng
`test_chase_app.py`	9	AppTest cho điều khiển và trạng thái ván
`test_app.py`	7	Luồng giao diện định tuyến
`test_app_modes.py`	7	Chuyển chế độ và bảo toàn session state
`test_shortest_path.py`	7	Dijkstra, đa cạnh, vô hướng tới đích, oracle ngẫu nhiên
`test_heuristics.py`	3	Nearest-neighbor và 2-opt
`test_preload.py`, `test_pbf_preload.py`	2	An toàn khi chuẩn bị dữ liệu
Tổng	**112**	**Tất cả đạt**

Lệnh `python -m unittest discover -s tests -q` chạy 112 test trong 18,431 giây và trả về `OK`. Một số test cố ý đưa metadata hoặc Pickle sai để kiểm tra đường lỗi; traceback log trong các trường hợp này là dữ liệu test, không phải test thất bại.

5.2.2 Các nhóm trường hợp biên

Đối với Dijkstra, test bao phủ nguồn bằng đích, đích không tới được, cạnh song song, graph có hướng, trọng số sai và so sánh với NetworkX trên graph ngẫu nhiên. Đối với Held–Karp, test bao phủ $n = 1$, tuyến mở, điểm cuối cố định, chu trình, ma trận bất đối xứng, chi phí vô hạn và đối chiếu vét cạn ở kích thước nhỏ.

Đối với trò chơi, các ca quan trọng gồm: bắt và thoát cùng nút thì bắt ưu tiên; thoát ở lượt cuối vẫn thắng trước timeout; máy bắt sau khi di chuyển; không cho đi sau terminal; loại nút đội máy khỏi niềm tin khi người trốn sống; policy nhận cùng quan sát phải cho cùng quyết định dù trạng thái ẩn khác; và mọi bước đi không vượt ngân sách. Các test này gắn trực tiếp với bất biến, không chỉ với dòng hiển thị.

5.3 Smoke test định tuyến trên graph thật

Kịch bản offline dùng năm địa điểm: Đại học Phenikaa, Sân vận động Mỹ Đình, Công viên Cầu Giấy, đường Thanh Niên bên Hồ Tây và Đinh Tiên Hoàng bên Hồ Hoàn Kiếm. Khoảng cách snap lớn nhất từ tọa độ mẫu tới graph là 217,85 m. Cùng một tập điểm được tối ưu theo hai trọng số.

Bảng 5.2: Kết quả smoke test năm địa điểm ngày 24/10/2026

Mục tiêu	Thứ tự chỉ số	Quãng đường	Thời gian	Dijkstra	Held–Karp
Chiều dài	0–1–2–4–3	26,276 km	32,59 phút	1,028 s	0,174 ms
Thời gian	0–1–2–4–3	27,049 km	29,93 phút	0,921 s	0,136 ms

Graph nạp trong 9,845 giây ở lần chạy này. Cả hai mục tiêu cho cùng thứ tự ghé, nhưng đường chi tiết khác: tối ưu thời gian dài hơn 0,773 km và nhanh hơn 2,66 phút theo trọng số ước tính. Điều này cho thấy tiêu chí tác động tới việc chọn đoạn đường ngay cả khi thứ tự điểm không đổi.

Thời gian Held–Karp dưới 0,2 ms nhỏ hơn nhiều so với Dijkstra gần một giây. Kết quả phù hợp với kiến trúc: chỉ có năm điểm trong bài toán quy hoạch động, trong khi Dijkstra làm việc trên graph hàng trăm nghìn nút. Không nên suy rộng thời gian này cho 10 điểm hay cho phần cứng khác mà không đo lại.

5.4 So sánh thuật toán chính xác và heuristic trên tầng thứ tự ghé

5.4.1 Thiết kế phép đo

Smoke test ở mục trước chỉ dùng năm địa điểm và một lần chạy, nên chưa đủ để nói về hai điều mà Chương 2 đã tuyên bố: Held–Karp tăng theo 2^n, còn hai heuristic không có bảo đảm tối ưu. Vì vậy nhóm bổ sung một benchmark riêng cho tầng thứ tự ghé, chạy bằng `scripts/benchmark_tsp.py`.

Thiết kế như sau. Mười một địa điểm đầu trong `POPULAR_LOCATIONS` được snap vào graph Hà Nội đóng gói, khoảng cách snap lớn nhất là 263,22 m. Ma trận chi phí có hướng đầy đủ 11×11 được tính một lần; với mỗi n từ 4 đến 11, phép đo dùng ma trận con $n \times n$ của chính ma trận đó. Nhờ vậy ba thuật toán nhận đúng cùng một dữ liệu đầu vào và phần Dijkstra không lẫn vào thời gian của tầng tổ hợp. Mỗi cấu hình được chạy lặp năm lần; bảng báo thời gian trung bình.

Với mỗi n, cả ba kiểu kết thúc đều được đo. Ở kiểu điểm cuối cố định, điểm cuối luôn là điểm cuối cùng của tập con. Khoảng cách chất lượng của một heuristic được định nghĩa là

$$\text{gap} = \left(\frac{C_{\text{heuristic}}}{C_{\text{Held–Karp}}} - 1 \right) \times 100\%,$$

hợp lệ vì Held–Karp là tối ưu chính xác trên cùng ma trận, nên mẫu số chính là chi phí nhỏ nhất.

5.4.2 Chất lượng nghiệm

Bảng 5.3: Chi phí nghiệm theo mục tiêu quãng đường; gap so với Held–Karp

Điểm	Kiểu kết thúc	Held–Karp (km)	NN (km)	Gap NN (%)	NN + 2-opt (km)	Gap 2-opt (%)
4	Tuyến mở	22,394	22,394	0,00	22,394	0,00
4	Điểm cuối cố định	22,394	22,394	0,00	22,394	0,00
4	Chu trình	37,068	37,081	0,03	37,068	0,00
5	Tuyến mở	26,276	26,509	0,89	26,276	0,00
5	Điểm cuối cố định	26,509	26,509	0,00	26,509	0,00
5	Chu trình	40,343	41,417	2,66	40,343	0,00
6	Tuyến mở	33,757	33,757	0,00	33,757	0,00
6	Điểm cuối cố định	33,757	33,757	0,00	33,757	0,00
6	Chu trình	52,566	54,806	4,26	52,566	0,00
7	Tuyến mở	35,384	35,384	0,00	35,384	0,00
7	Điểm cuối cố định	40,927	42,206	3,13	40,927	0,00
7	Chu trình	53,545	56,433	5,39	54,369	1,54
8	Tuyến mở	35,384	35,384	0,00	35,384	0,00
8	Điểm cuối cố định	41,171	43,992	6,85	41,171	0,00
8	Chu trình	53,981	56,433	4,54	54,369	0,72
9	Tuyến mở	40,905	40,905	0,00	40,905	0,00
9	Điểm cuối cố định	43,612	43,612	0,00	43,612	0,00
9	Chu trình	56,087	61,954	10,46	56,087	0,00
10	Tuyến mở	45,946	45,946	0,00	45,946	0,00
10	Điểm cuối cố định	49,580	53,370	7,64	49,580	0,00
10	Chu trình	58,153	66,995	15,20	60,359	3,79
11	Tuyến mở	47,056	47,056	0,00	47,056	0,00
11	Điểm cuối cố định	55,512	66,351	19,53	57,718	3,97
11	Chu trình	59,368	68,105	14,72	61,573	3,71
Trung bình 24 cấu hình				**3,97**		**0,57**

Điểm	Kiểu kết thúc	Held–Karp (km)	NN (km)	Gap NN (%)	NN + 2-opt (km)	Gap 2-opt (%)
Lớn nhất				**19,53**		**3,97**

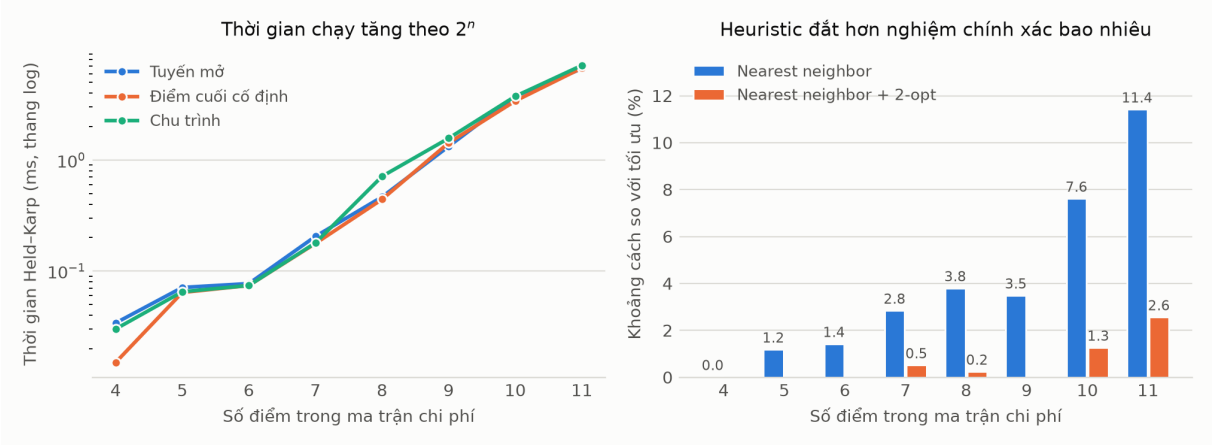
Kết quả cho ba nhận định. Thứ nhất, heuristic **không** sai một cách hiển nhiên: ở tuyến mở, nearest-neighbor trùng nghiệm tối ưu trong cả tám kích thước. Nguyên nhân là tuyến mở không bị ràng buộc điểm cuối, nên lựa chọn tham lam "đi tiếp tới điểm gần nhất" phù hợp với cấu trúc địa lý trải dài của tập điểm mẫu.

Thứ hai, khoảng cách xuất hiện ngay khi có thêm ràng buộc. Với chu trình và điểm cuối cố định, nearest-neighbor lệch tới 15,20% và 19,53%. Đây đúng là điểm yếu đã nêu trong lý thuyết: quyết định tham lam ban đầu không tính tới chi phí quay về hoặc chi phí phải kết thúc tại một điểm định sẵn [10].

Thứ ba, 2-opt thu hẹp phần lớn khoảng cách nhưng không xóa được: gap trung bình giảm từ 3,97% xuống 0,57%, còn gap lớn nhất giảm từ 19,53% xuống 3,97%. Trong 17/24 cấu hình, 2-opt chạm đúng nghiệm tối ưu, nhưng 7 cấu hình còn lại vẫn kẹt ở cực tiểu cục bộ. Quan sát này là bằng chứng trực tiếp cho phát biểu "heuristic không có bảo đảm tối ưu toàn cục" ở mục 2.3.2.

Với mục tiêu thời gian di chuyển, xu hướng tương tự nhưng rõ hơn ở phía nearest-neighbor: gap trung bình 5,89% và gap lớn nhất 28,22% tại cấu hình 11 điểm, điểm cuối cố định. Trên bộ dữ liệu này 2-opt đạt 0,00% ở cả 24 cấu hình. Số liệu đầy đủ nằm trong `output/routing/benchmark-tsp-time.json`.

5.4.3 Thời gian chạy và kiểm chứng độ phức tạp



Hình 5.1: Trái: thời gian Held–Karp theo số điểm trên thang log, ba kiểu kết thúc. Phải: khoảng cách trung bình so với nghiệm tối ưu của hai heuristic

Bảng 5.4: Thời gian trung bình của ba thuật toán theo số điểm (kiểu chu trình)

Điểm	Số trạng thái DP $(n-1)2^{n-1}$	Held–Karp (ms)	NN (ms)	NN + 2-opt (ms)	Tỉ lệ tăng Held–Karp
4	24	0,030	0,008	0,019	–
5	64	0,065	0,009	0,039	2,17
6	160	0,074	0,005	0,029	1,14
7	384	0,179	0,007	0,041	2,42
8	896	0,712	0,012	0,075	3,98
9	2048	1,569	0,012	0,080	2,20
10	4608	3,748	0,011	0,093	2,39
11	10240	7,113	0,013	0,115	1,90

Cột cuối là tỉ số thời gian giữa hai kích thước liên tiếp. Trung bình nhân của các tỉ số từ 7 đến 11 điểm là khoảng 2,4 lần cho mỗi điểm thêm vào, nằm quanh mức mà công thức $O(n^2 2^n)$ dự đoán ở dải này. Hai giá trị nhỏ ở $n = 5$ và $n = 6$ là nhiễu đo: ở mức vài chục micro giây, chi phí khởi tạo và dao động bộ đệm lấn át phần tính toán, nên không nên đọc tỉ lệ tăng tại các kích thước nhỏ nhất.

Điều quan trọng về mặt kỹ thuật là mức tuyệt đối. Ở giới hạn thiết kế 10 điểm cần ghé (11 điểm trong ma trận), Held–Karp vẫn chỉ tốn khoảng 7 ms, nhỏ hơn hai bậc độ lớn so với bước lập ma trận chi phí. Nói cách khác, trong phạm vi mà giao diện cho phép, tầng tổ hợp **không** phải nút cổ chai; chi phí thực tế nằm ở Dijkstra trên graph lớn.

Bảng 5.5: Thời gian lập ma trận chi phí bằng Dijkstra theo số điểm

Điểm	Số lần chạy Dijkstra	Thời gian (giây)
4	4	0,484
5	5	0,806
6	6	1,462
7	7	1,618
8	8	1,649
9	9	1,860
10	10	1,995
11	11	2,218

Thời gian Dijkstra tăng gần tuyến tính theo số điểm chứ không theo giai thừa, vì mỗi điểm chỉ thêm một lần chạy nguồn đơn nhiều đích. Tỉ lệ giữa hai tầng tại 11 điểm là khoảng $2{,}218$ giây so với $7{,}1$ ms, tức tầng đồ thị chiếm hơn 99,6% tổng thời gian tối ưu. Đây là lý do kiến trúc tách hai tầng và cache graph trong phiên: muốn giảm thời gian chờ thì phải nhắm vào Dijkstra và việc nạp graph, không phải vào Held–Karp.

5.4.4 Diễn giải và giới hạn của phép đo

Benchmark này cho phép kết luận ba điều trong phạm vi dữ liệu đã dùng: Held–Karp khả thi về thời gian ở toàn bộ dải được hỗ trợ; nearest-neighbor có thể lệch hai chữ số phần trăm khi tuyến bị ràng buộc điểm cuối; và 2-opt là một cải thiện rẻ nhưng không phải một lời bảo đảm.

Giới hạn cần nói rõ: phép đo chỉ dùng một tập 11 địa điểm tại Hà Nội và các tập con lồng nhau của nó, nên không phải một mẫu ngẫu nhiên của mọi cấu hình điểm. Với tập điểm cụm chặt hoặc phân bố hình sao, thứ hạng heuristic có thể khác. Thời gian đo

trên một máy phát triển với Python 3.11 và không loại bỏ ảnh hưởng của tải hệ thống; vì vậy tỉ lệ giữa các tầng đáng tin hơn giá trị tuyệt đối. Cuối cùng, phép đo tách ma trận chi phí ra khỏi tầng tổ hợp một cách có chủ ý, nên các con số ở Bảng 5.4 không cộng lại thành thời gian người dùng cảm nhận; Bảng 5.5 mới là phần chiếm đa số thời gian đó.

5.5 Thiết kế benchmark trò chơi

Mỗi sân dùng 20 seed, ba bot người trốn và ba chiến thuật máy, tạo $20 \times 3 \times 3 = 180$ ván/sân và 360 ván tổng cộng. Bộ luật cố định: $b_r = b_h = 160$ m, tiết lộ mỗi ba lượt, tối đa 18 lượt. Ba bot người trốn là:

- `random`: chọn ngẫu nhiên trong các nút khả đạt;
- `nearest_exit`: ưu tiên tiến về lối thoát gần nhất;
- `evasive`: ưu tiên tránh đội máy trong khi vẫn tìm lối thoát.

Bot và vị trí khởi đầu đều cố định theo seed. Vì không có người chơi thật, phép đo chỉ đánh giá tương tác với ba hành vi lập trình sẵn. Không chiến thuật nào trong hai phía được chứng minh tối ưu.

5.6 Kết quả trên sân lưới mô phỏng

Bảng 5.6: Benchmark sân lưới 81 nút; mỗi dòng gồm 20 ván

Bot trốn	Máy	Thoát	Bắt	Hết lượt	Lượt TB	ms/bước TB	ms max
Ngẫu nhiên	Dấu vết	3	16	1	6,15	0,724	3,470
Ngẫu nhiên	Tìm kiếm	2	18	0	4,45	0,882	3,523
Ngẫu nhiên	Phối hợp	1	19	0	4,80	3,973	14,897
Gần lối thoát	Dấu vết	18	2	0	3,15	0,628	2,806
Gần lối thoát	Tìm kiếm	19	1	0	3,20	0,870	2,828
Gần lối thoát	Phối hợp	13	7	0	3,20	2,972	10,620
Né tránh	Dấu vết	19	1	0	3,90	0,589	3,147
Né tránh	Tìm kiếm	20	0	0	3,70	1,119	3,345
Né tránh	Phối hợp	19	1	0	4,50	3,304	11,581

Trên bot ngẫu nhiên, phối hợp bắt 19/20 ván, cao hơn dấu vết 16/20. Trên bot đi thẳng tới lối thoát, phối hợp bắt 7/20, trong khi hai chiến thuật còn lại chỉ bắt 2/20 và 1/20. Đây là bằng chứng cấu hình cụ thể cho lợi ích của thành phần chặn lối thoát. Tuy nhiên bot né tránh vẫn thoát 19/20 trước phối hợp. Tổng trên 60 ván/chiến thuật, phối hợp bắt 27 ván, còn dấu vết và tìm kiếm cùng bắt 19 ván.

Đổi lại, phối hợp tốn 2,972–3,973 ms/bước trung bình, cao hơn các chính sách đơn dưới 1,2 ms ở phần lớn cấu hình. Mức này vẫn phù hợp tương tác, nhưng phản ánh chi phí xét tổ hợp ứng viên giữa hai đội.

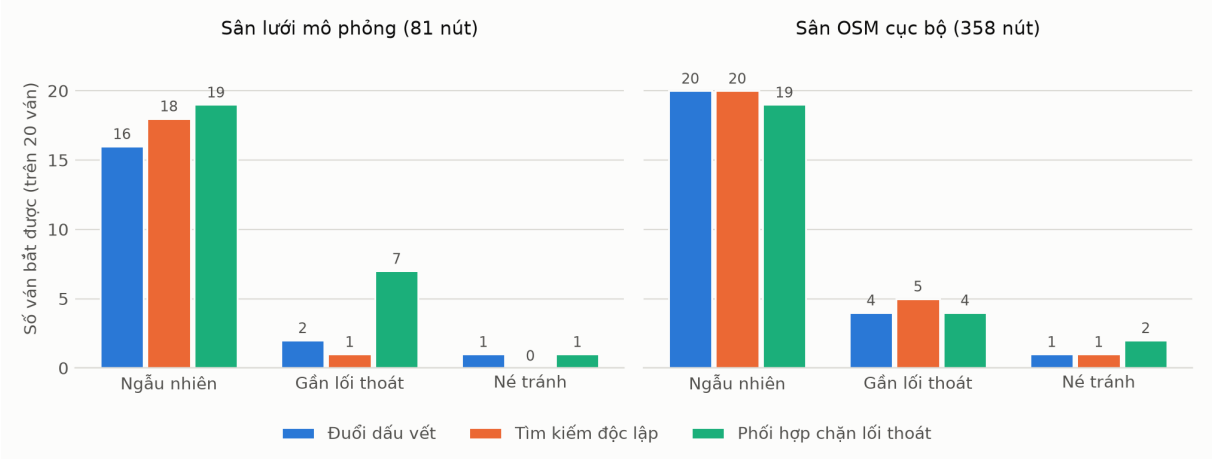
5.7 Kết quả trên sân OSM cục bộ

Bảng 5.7: Benchmark sân OSM 358 nút; mỗi dòng gồm 20 ván

Bot trốn	Máy	Thoát	Bắt	Hết lượt	Lượt TB	ms/bước TB	ms max
Ngẫu nhiên	Dấu vết	0	20	0	8,25	2,026	11,804
Ngẫu nhiên	Tìm kiếm	0	20	0	7,95	2,580	10,702
Ngẫu nhiên	Phối hợp	0	19	1	9,30	5,241	18,427
Gần lối thoát	Dấu vết	16	4	0	6,30	2,137	11,864
Gần lối thoát	Tìm kiếm	15	5	0	5,95	2,297	8,666
Gần lối thoát	Phối hợp	16	4	0	6,15	4,255	12,369
Né tránh	Dấu vết	15	1	4	9,65	1,816	8,055
Né tránh	Tìm kiếm	15	1	4	9,90	2,273	8,620
Né tránh	Phối hợp	13	2	5	9,85	3,716	11,151

Trên sân OSM, cả ba chiến thuật bắt toàn bộ hoặc gần toàn bộ bot ngẫu nhiên. Với bot gần lối thoát, số ván bắt chỉ từ 4 đến 5; với bot né tránh, chỉ từ 1 đến 2, còn 4–5 ván kết thúc do hết lượt. Tổng số bắt trên 60 ván lần lượt là 25, 26 và 25 cho dấu vết, tìm kiếm và phối hợp. Vì vậy benchmark này bác bỏ cách diễn giải đơn giản rằng phối hợp luôn mạnh nhất.

Sự khác biệt giữa sân lưới và OSM có thể do topology, vị trí lối thoát, đường cụt và shortlist ứng viên, nhưng benchmark hiện tại chưa cô lập nguyên nhân. Đây là giả thuyết cần thí nghiệm ablation, không phải kết luận đã kiểm chứng.

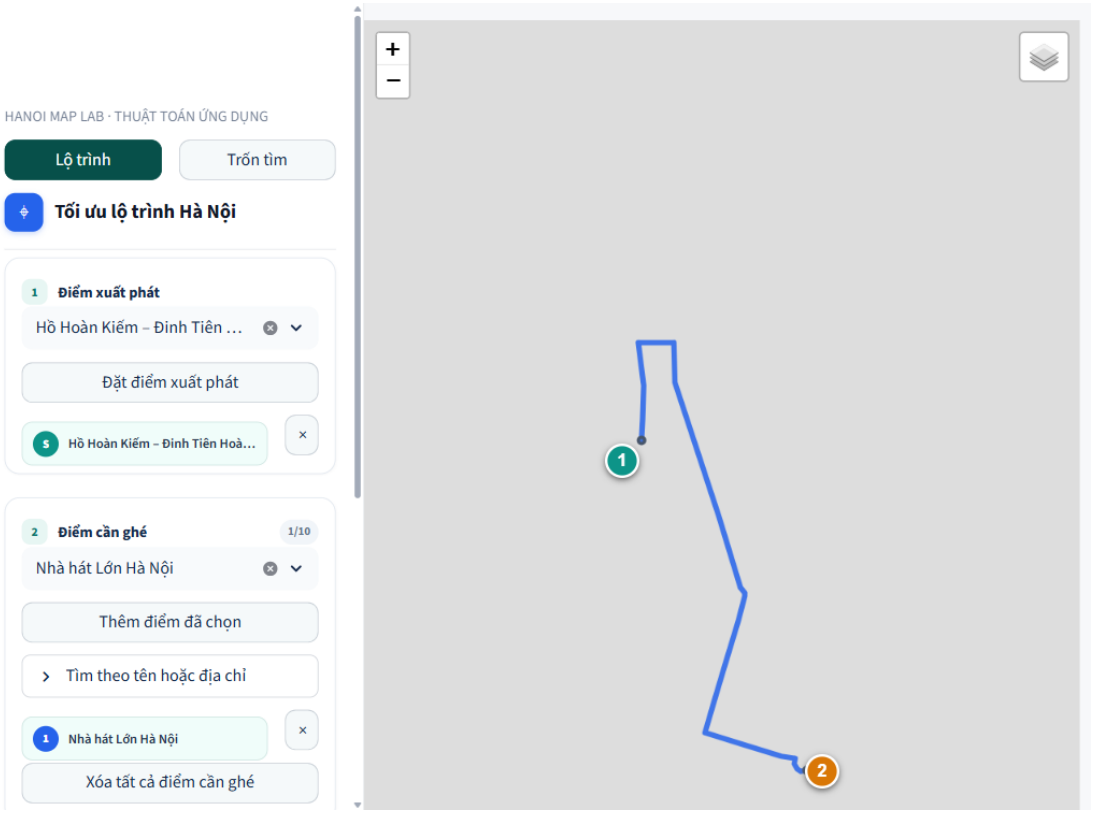


Hình 5.2: Số ván bắt được trên mỗi cấu hình 20 ván, đối chiếu hai sân. Thứ hạng ba chiến thuật đảo khi đổi sân và đổi bot người trốn

Hình 5.2 đặt hai sân cạnh nhau để thấy rõ điều mà hai bảng số rời rạc dễ che: lợi thế của chiến thuật phối hợp chỉ xuất hiện ở sân lưới và chỉ với hai trong ba bot, còn trên sân OSM ba cột gần như bằng nhau ở mọi nhóm. Nếu một chiến thuật thực sự trội, các cột xanh lục phải cao hơn ở cả sáu nhóm; dữ liệu không cho thấy điều đó.

5.8 Đánh giá giao diện và tích hợp

AppTest kiểm tra mở mặc định ở Lộ trình, deep link tới Trốn tìm, chuyển qua lại và giữ tiêu chí/kiểu điểm cuối/click bản đồ. Các kịch bản game đi đến ba kết cục `escaped`, `caught`, `timeout`. Playwright được dùng để kiểm tra trình duyệt thật: chọn Hồ Hoàn Kiếm tới Nhà hát Lớn, tối ưu được tuyến 1,36 km với thời gian hiển thị 1,4 phút; chuyển sang trò chơi rồi quay lại vẫn giữ nguyên kết quả.



Hình 5.3: Tuyến thực được vẽ sau khi tối ưu trong giao diện tích hợp

Trong ảnh kiểm tra, tile nền màu xám do sự cố phân giải/chính sách mạng của máy thử nghiệm, nhưng polyline và chỉ số thuật toán vẫn hiển thị. Điều này minh họa việc tile nền và graph định tuyến là hai thành phần độc lập.

5.9 Các mối đe dọa tới tính hợp lệ

Tính hợp lệ nội tại: benchmark dùng cùng seed giữa các chiến thuật, nhưng không thực hiện kiểm định thống kê hoặc hoán đổi ngẫu nhiên thứ tự chạy. Thời gian có thể bị ảnh hưởng bởi warm cache và tải hệ thống.

Tính hợp lệ cấu trúc: tỉ lệ bắt/thoát đo hiệu quả trong luật trò chơi đã định nghĩa, không đo chất lượng chiến thuật trong pursuit–evasion liên tục. Hàm coverage là proxy khoảng cách, không phải xác suất bắt.

Tính hợp lệ ngoại suy: chỉ có hai sân, ba bot và 20 seed. Sân OSM là phần graph cục bộ đã điều chỉnh hai chiều. Không thể suy ra hiệu quả trên toàn Hà Nội, người chơi thật hoặc điều kiện giao thông thực tế.

Tính đúng dữ liệu: trọng số thời gian là ước tính tĩnh. Kết quả 29,93 phút không phải dự báo thời gian thực và không nên dùng như chỉ dẫn giao thông.

CHƯƠNG 6

MINH CHỨNG SẢN PHẨM VÀ KỊCH BẢN DEMO

6.1 Khởi chạy ứng dụng

Từ PowerShell tại thư mục dự án, dùng môi trường Python 3.11 đã cài dependency:

Listing 6.1: Lệnh khởi chạy ứng dụng

```
.\.venv\Scripts\python.exe -m streamlit run app.py --server.port 8502
```

Mở `http://localhost:8502`. Mặc định ứng dụng vào chế độ Lộ trình. Query `?mode=chase` mở trực tiếp chế độ Trốn tìm. Graph định tuyến lớn cần thời gian nạp lần đầu; các lần tương tác sau dùng cache.

6.2 Kịch bản demo chế độ Lộ trình

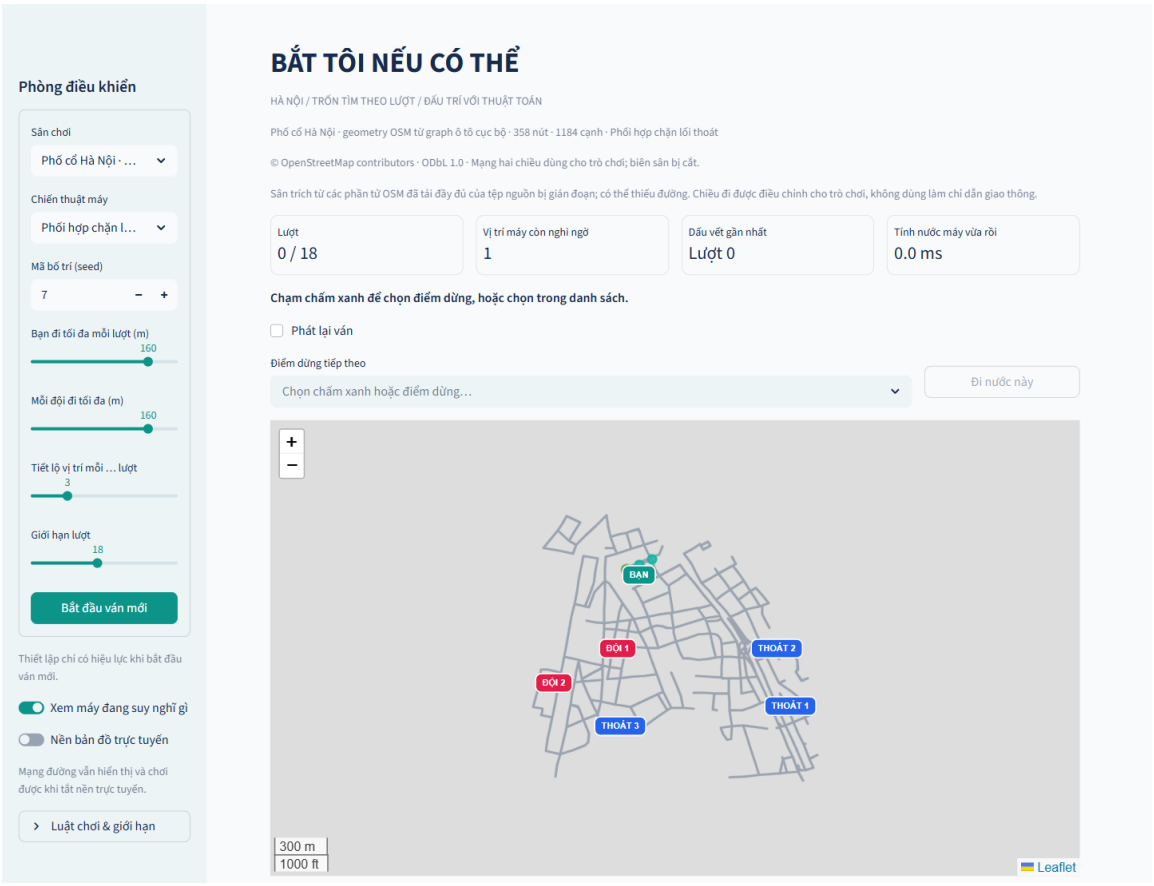
Kịch bản đề xuất dài khoảng ba phút:

1. Chọn Đại học Phenikaa làm điểm xuất phát; thêm Mỹ Đình, Cầu Giấy, Hồ Tây và Hồ Hoàn Kiếm làm điểm cần ghé.
2. Chọn mục tiêu *Quãng đường*, điểm cuối tự động và bấm tối ưu. Giải thích pipeline Dijkstra $\rightarrow$ ma trận chi phí $\rightarrow$ Held–Karp $\rightarrow$ khôi phục tuyến.
3. Đọc thứ tự ghé, tổng km, thời gian ước tính và thời gian từng thuật toán.
4. Chuyển mục tiêu sang *Thời gian di chuyển*; chỉ ra rằng tuyến có thể dài hơn nhưng thời gian ước tính thấp hơn.
5. Chọn điểm cuối cố định hoặc quay về điểm đầu để minh họa ba biến thể.

Khi trình bày, cần nói rõ Held–Karp tối ưu chính xác trên ma trận chi phí của các điểm đã chọn, không phải giải mọi biến thể VRP. Tile nền không tham gia phép tính.

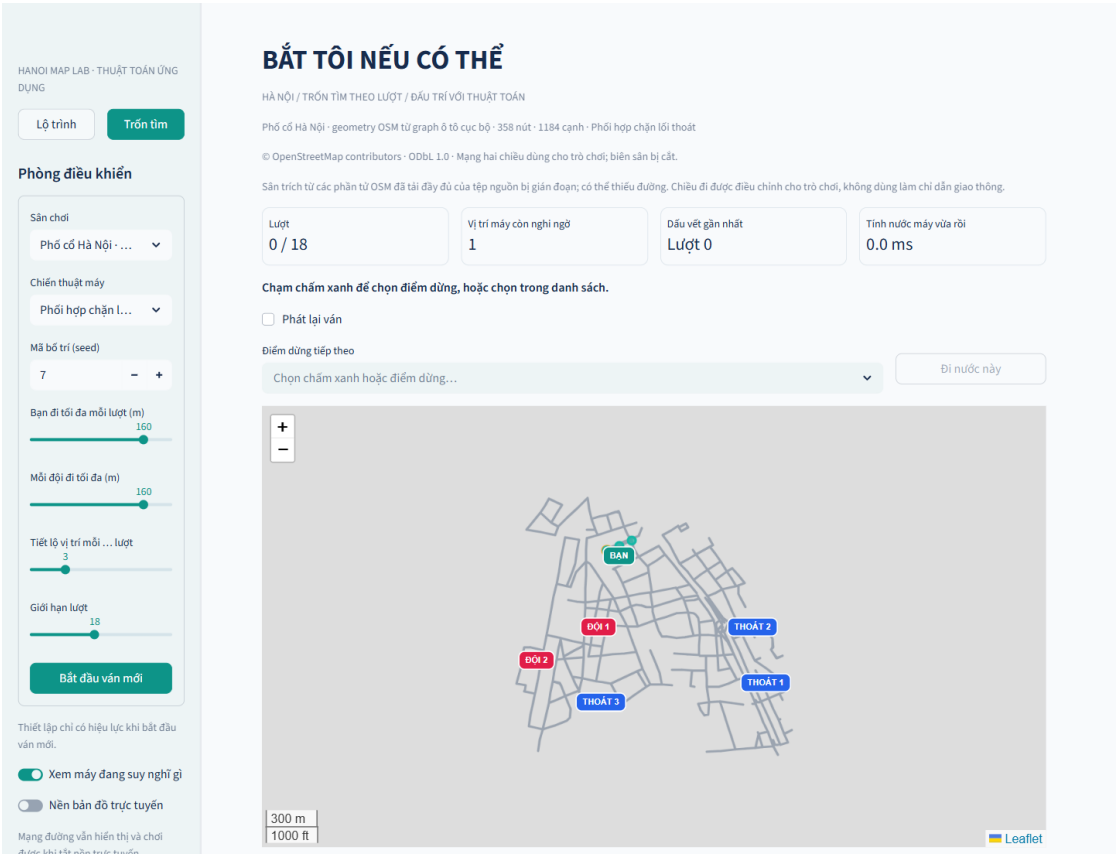
6.3 Kịch bản demo chế độ Trốn tìm

Nhấn *Trốn tìm*, chọn sân phố Hà Nội, chiến thuật *Phối hợp chặn lối thoát*, seed 7 và giữ bộ luật mặc định. Bắt đầu ván mới; giải thích màu xanh lá là người chơi, đỏ là đội máy, xanh dương là lối thoát, các chấm xanh nhạt là điểm đến hợp lệ.



Hình 6.1: Phòng điều khiển cùng trạng thái đầu ván trên sân phố cổ Hà Nội: bộ luật đang dùng nằm ở cột trái, bốn chỉ số trạng thái nằm trên bản đồ. Ở lượt 0, vị trí người trốn vừa được tiết lộ nên "Vị trí máy còn nghi ngờ" bằng 1

Bảng điều khiển ở Hình 6.1 là nơi nên dừng lại khi trình bày: mọi tham số của bộ luật đều hiện rõ, nên người nghe kiểm chứng được rằng kết quả ván chơi phụ thuộc vào cấu hình chứ không phải một hộp đen. Lưu ý thiết lập chỉ có hiệu lực khi bắt đầu ván mới, vì vậy cần chốt thông số trước khi bấm nút.



Hình 6.2: Giao diện trò chơi trên graph phố Hà Nội cục bộ

Chơi ít nhất ba lượt để xuất hiện một lần tiết lộ. Trước lượt tiết lộ, chỉ ra số "Vị trí máy còn nghi ngờ" tăng theo reachability; ở lượt tiết lộ, tập này co về một nút. Bật tùy chọn xem giải thích để đọc mục tiêu mà máy chọn. Sau đó bật phát lại, kéo slider qua các lượt và tải JSON lịch sử.

6.4 Thông điệp thuật toán cần nhấn mạnh

Bảng 6.1: Thông điệp bảo vệ đồ án và bằng chứng tương ứng

Thông điệp	Bằng chứng có thể trình diễn
Dijkstra giải đường đi giữa cặp điểm trên graph có hướng	Polyline chi tiết, ma trận chi phí và test oracle ngẫu nhiên
Held–Karp là tối ưu chính xác trong miền nhỏ	So sánh với vét cạn ở test; thời gian dưới 0,2 ms trong smoke test 5 điểm
Heuristic không có bảo đảm tối ưu	Bảng benchmark cho thấy thứ hạng thay đổi theo bot và sân
Máy không biết vị trí ẩn	API `Observation` và test hai trạng thái ẩn có cùng quan sát
Giao diện không thay thế kiểm thử	112 unit/integration test cùng kiểm tra trình duyệt thật

6.5 Tình huống lỗi và phương án phục hồi

Nếu tile nền không tải, chuyển layer vệ tinh nếu khả dụng hoặc tiếp tục demo graph/marker; không cần tải lại dữ liệu định tuyến. Nếu graph lớn chưa nạp được, chuyển sang Trốn tìm vì sân nhỏ độc lập, sau đó kiểm tra Git LFS. Nếu thao tác game bị chọn sai nút, dùng dropdown điểm đến thay cho click. Nếu demo cần tái lập, giữ nguyên seed và thông số trước khi bấm bắt đầu ván mới.

Giới hạn cần nói công khai khi bảo vệ

Sân OSM của trò chơi chỉ là một phần mạng đường và đã chuyển hai chiều; trọng số thời gian của định tuyến là ước tính tĩnh; benchmark dùng bot viết sẵn; chiến thuật phối hợp là heuristic. Các giới hạn này không làm mất giá trị đồ án, nhưng quyết định phạm vi mà kết quả có thể hỗ trợ.

CHƯƠNG 7

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết quả đạt được

Đồ án đã hoàn thành một ứng dụng thuật toán đồ thị gồm hai chế độ tích hợp. Phần định tuyến xử lý graph ô tô Hà Nội quy mô 350.930 nút và 864.982 cạnh, hỗ trợ tối đa 10 điểm ghé, hai hàm mục tiêu và ba kiểu kết thúc. Phần thuật toán tách rõ Dijkstra cho đường đi giữa cặp điểm, Held–Karp cho thứ tự ghé chính xác và hai heuristic để đối chiếu. Benchmark 24 cấu hình xác nhận cả hai mặt của cách tách này: Held–Karp giữ thời gian dưới 7,2 ms ở giới hạn 10 điểm ghé, còn heuristic đổi bảo đảm tối ưu lấy tốc độ với cái giá đo được là tới 19,53% chi phí dư thừa khi tuyến bị ràng buộc điểm cuối.

Phần trò chơi mở rộng bài toán từ tối ưu tĩnh sang ra quyết định tuần tự dưới thông tin không đầy đủ. Tập niềm tin, chu kỳ tiết lộ và ranh giới `Observation` làm rõ máy biết gì ở mỗi lượt. Ba chiến thuật có mức phức tạp và hành vi khác nhau, được so sánh trên 360 ván tái lập.

Bộ 112 test đều đạt trong lần chạy ngày 24/10/2026. Smoke test và kiểm tra trình duyệt chứng minh pipeline hoạt động trên graph thật và trạng thái được giữ khi chuyển chế độ. Sản phẩm có replay, log JSON và thông báo giới hạn dữ liệu để hỗ trợ trình diễn và kiểm tra.

7.2 Bài học thuật toán

Thứ nhất, một bài toán ứng dụng thường cần ghép nhiều tầng tối ưu. Dijkstra đúng cho từng đoạn chưa quyết định được thứ tự ghé; Held–Karp giải tầng tổ hợp sau khi ma trận chi phí được tạo. Thứ hai, thuật toán chính xác và heuristic có vai trò khác nhau: chính xác cho bảo đảm trong miền nhỏ, heuristic đổi bảo đảm lấy tốc độ hoặc hành vi thực dụng. Thứ ba, trong hệ thống thông tin ẩn, định nghĩa giao diện quan sát quan trọng không kém hàm đánh giá; nếu policy đọc được trạng thái thật, mọi kết quả chiến thuật trở nên vô nghĩa.

Cuối cùng, kiểm thử cần bám vào giả định và bất biến. Kiểm tra một nút bấm hoạt động không chứng minh thuật toán đúng; ngược lại, unit test không chứng minh giao diện sử dụng được. Đồ án kết hợp oracle, test biên, AppTest, smoke test và trình duyệt thật để giảm khoảng cách này.

7.3 Hạn chế

- Thời gian di chuyển là tĩnh, chưa có dữ liệu ùn tắc theo thời điểm.
- Benchmark tầng thứ tự ghé chỉ dùng một tập 11 địa điểm và các tập con lồng nhau, chưa lấy mẫu ngẫu nhiên trên nhiều dạng phân bố điểm.
- Held–Karp tăng theo 2^n, nên giới hạn hiện tại là 10 điểm ghé.
- Hệ thống chưa giải VRP nhiều xe, time window, capacity hoặc điểm động [2].
- Graph trò chơi OSM không đầy đủ và được điều chỉnh hai chiều.
- Bắt chỉ xét vị trí nút sau bước đi; chưa xét hai tác nhân giao nhau giữa cạnh.
- Tập niềm tin không mang xác suất; hệ số 0,35 và shortlist 8 ứng viên chưa được hiệu chỉnh độc lập.
- Benchmark chỉ có ba bot, hai sân và 20 seed; chưa có người chơi thật hoặc kiểm định thống kê.
- Lớp tile trực tuyến phụ thuộc DNS và chính sách nhà cung cấp, dù thuật toán cục bộ không phụ thuộc lớp nền.

7.4 Hướng phát triển có thứ tự ưu tiên

Ưu tiên 1 – củng cố đánh giá. Mở rộng seed và topology, thêm baseline ngẫu nhiên, chạy ablation hệ số lối thoát/shortlist/chu kỳ tiết lộ, báo khoảng tin cậy và thu thập một tập ván người chơi. Tiêu chí tiếp tục là kết quả lặp lại qua nhiều sân; nếu ưu thế chỉ xuất hiện trên lưới, cần thu hẹp claim thay vì đổi tên chiến thuật.

Ưu tiên 2 – xác suất hóa niềm tin. Thay tập B_t bằng phân phối $p_t(v)$ với mô hình chuyển của người trốn, sau đó đánh giá expected capture hoặc information gain. Cần có positive control: khi mô hình chuyển đúng với bot sinh dữ liệu, phương pháp xác suất phải cải thiện so với uniform belief; nếu không, cơ chế chưa được hỗ trợ.

Ưu tiên 3 – lập kế hoạch nhiều bước. So sánh heuristic một bước với beam search hoặc Monte Carlo tree search [17] trên sân nhỏ. Trước khi mở rộng, cần đo trade-off giữa

tỉ lệ bắt và độ trễ; không gọi là tối ưu nếu chưa có chứng minh hoặc oracle kích thước nhỏ.

Ưu tiên 4 – định tuyến động và bền vững. Nếu có dữ liệu thời gian theo cạnh $c_{ij}(t)$, mô hình phải kiểm tra thuộc tính FIFO và chạy thuật toán phù hợp; không dùng một hệ số "giờ cao điểm" toàn cục rồi gọi đó là time-dependent routing. Một hướng gần hơn là cho phép chặn một số cạnh và đo độ tăng chi phí, tạo kịch bản "phá đường–cứu tuyến" có đối chứng rõ ràng.

7.5 Kết luận chung

Hệ thống cho thấy một đề tài tối ưu lộ trình truyền thống có thể phát triển thành một phòng thí nghiệm thuật toán đồ thị có tính tương tác mà vẫn giữ được phần lõi ban đầu. Giá trị chính không nằm ở tuyên bố một chiến thuật luôn tốt nhất, mà ở việc mô hình hóa rõ, triển khai có ranh giới thông tin, so sánh đúng mức và cung cấp bằng chứng tái lập. Đây là nền tảng phù hợp để tiếp tục nghiên cứu các bài toán định tuyến và pursuit–evasion phức tạp hơn.

CHƯƠNG A

PHỤ LỤC TÁI LẬP

A.1 Cài đặt trên Windows 11

Listing A.1: Tạo môi trường và cài dependency

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
git lfs pull
```

Hai graph lớn được quản lý bằng Git LFS. Nếu tệp chỉ có vài dòng pointer, cần cài Git LFS và chạy lại `git lfs pull` trước khi đánh giá định tuyến.

A.2 Lệnh kiểm thử và benchmark

Listing A.2: Các lệnh tái lập kết quả chính

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -q
.\.venv\Scripts\python.exe scripts\smoke_offline.py
.\.venv\Scripts\python.exe scripts\benchmark_chase.py `
  --source demo --seeds 20 --output output\chase\benchmark-demo.json
.\.venv\Scripts\python.exe scripts\benchmark_chase.py `
  --source osm --seeds 20 --output output\chase\benchmark-osm.json
.\.venv\Scripts\python.exe scripts\benchmark_tsp.py --weight length
.\.venv\Scripts\python.exe scripts\benchmark_tsp.py --weight
   travel_time `
  --output output\routing\benchmark-tsp-time.json
```

PowerShell dùng dấu backtick để nối dòng. Trên shell khác, thay bằng ký tự nối dòng tương ứng hoặc viết mỗi lệnh trên một dòng.

A.3 Cấu trúc mã nguồn và quy mô

Listing A.3: Cây thư mục dự án (bỏ qua tệp sinh ra)

```
Hanoi_map_optimizer/
```

```
|-- app.py                        # diem vao Streamlit, chuyen che do
|-- chase_app.py                  # shortcut mo thang che do Tron tim
|-- data/
|   |-- popular_locations.py      # 13 dia diem mau da kiem tra toa do
|   |-- hanoi_urban_drive.pkl     # graph dinh tuyen (Git LFS)
|   |-- hanoi_urban_drive.graphml
|   `-- hanoi_urban_drive.meta.json
|-- src/
|   |-- shortest_path.py          # Dijkstra + ma tran chi phi
|   |-- tsp.py                    # Held-Karp cho ba kieu ket thuc
|   |-- heuristics.py             # nearest-neighbor, 2-opt
|   |-- route_optimizer.py        # dieu phoi pipeline dinh tuyen
|   |-- map_service.py            # nap/cache graph, kiem tra metadata
|   |-- chase.py                  # luat, Observation, ba chien thuat
|   |-- chase_maps.py             # san luoi, san OSM, sinh kich ban
|   |-- chase_ui.py               # dieu khien van, replay, xuat JSON
|   |-- visualization.py          # ve ban do Folium
|   |-- geocoding.py              # tim dia diem theo ten
|   |-- utils.py, constants.py    # kieu du lieu chung, hang so
|-- tests/                        # 112 ca kiem thu tren 11 tep
|-- scripts/
|   |-- smoke_offline.py          # smoke test tren graph dong goi
|   |-- benchmark_tsp.py          # benchmark tang thu tu ghe
|   |-- benchmark_chase.py        # benchmark 360 van tro choi
|   |-- make_report_figures.py    # sinh hinh bieu do cho bao cao
|   `-- preload_hanoi*.py         # chuan bi du lieu OSM
|-- output/                       # ket qua benchmark dang JSON
`-- report/                       # bao cao LaTeX nay
```

Bảng A.1: Quy mô mã nguồn theo nhóm tệp

Nhóm	Số tệp Python	Số dòng
Giao diện (`app.py`, `chase_app.py`)	2	745
Thuật toán và dịch vụ (`src/`)	13	1.425
Kiểm thử (`tests/`)	12	1.344
Script dữ liệu và benchmark (`scripts/`)	9	891
Dữ liệu điểm mẫu (`data/`)	2	19
Tổng	**38**	**4.424**

Tỉ lệ giữa mã kiểm thử và mã thuật toán xấp xỉ 0,94:1. Con số này không phải mục tiêu tự thân, nhưng cho thấy phần kiểm chứng được đầu tư tương đương phần triển khai.

A.4 Kết quả chạy bộ kiểm thử

Listing A.4: Kết xuất rút gọn của lần chạy ngày 24/10/2026

```
> .\.venv\Scripts\python.exe -m unittest discover -s tests -q

[... cac dong log cua test duong loi, vi du: ]
_pickle.UnpicklingError: invalid load key, 'n'.
2026-10-24 09:07:56.798 No runtime found, using
   MemoryCacheStorageManager
----------------------------------------------------------------------
Ran 112 tests in 18.431s

OK
```

Dòng `UnpicklingError` ở trên đến từ ca kiểm thử cố ý nạp một tệp Pickle hỏng để xác nhận hệ thống báo lỗi thân thiện thay vì sập. Traceback được in ra là dữ liệu của test, không phải test thất bại; kết luận nằm ở hai dòng cuối.

A.5 Lược đồ tệp kết quả benchmark

Mọi số liệu ở Chương 5 đều được lưu dưới dạng JSON để có thể kiểm tra lại mà không cần chạy lại. Hai lược đồ như sau.

Listing A.5: Lược đồ `output/routing/benchmark-tsp.json`

```
{
  "weight": "length",            # muc tieu toi uu
  "repeats": 5,                  # so lan lap moi cau hinh
  "graph_nodes": 350930,
  "graph_edges": 864982,
  "max_snap_distance_m": 263.22,
  "locations": [ ... ],          # ten 11 dia diem theo thu tu chi so
  "dijkstra_scaling": [ {"points": 4, "dijkstra_seconds": 0.484}, ...
     ],
  "results": [
    {
      "points": 11, "destinations": 10, "route_mode": "fixed_end",
      "states": 10240,           # (n-1) * 2^(n-1)
      "exact_order": [0, 9, 8, ...],
      "unit": "km",
      "exact_cost": 55.512,      # nghiem Held-Karp
      "nn_cost": 66.351, "nn_gap_percent": 19.53, "nn_mean_ms": 0.012,
      "two_opt_cost": 57.718, "two_opt_gap_percent": 3.97,
      "held_karp_mean_ms": 6.736, "held_karp_min_ms": 6.2
    }
  ],
  "status": "OK"
}
```

Listing A.6: Lược đồ `output/chase/benchmark-*.json`

```
{
  "source": "demo",              # "demo" (san luoi) hoac "osm"
  "nodes": 81, "edges": 288,
  "rules": {"runner_budget": 160, "hunter_budget": 160,
            "reveal_every": 3, "max_turns": 18},
  "seeds": [0, 1, ..., 19],
  "results": [
    {
      "runner": "nearest_exit",  # random | nearest_exit | evasive
```

```
      "hunter": "contain",        # trail | search | contain
      "games": 20,
      "escaped": 13, "caught": 7, "timeout": 0,
      "mean_turns": 3.2,
      "mean_step_ms": 2.972, "max_step_ms": 10.62
    }
  ]
}
```

Bất biến kiểm tra được trên mỗi dòng kết quả là `escaped + caught + timeout = games`. Nếu tổng không khớp, hoặc benchmark dừng giữa chừng, hoặc một kết cục chưa được phân loại.

A.6 Sinh lại hình vẽ của báo cáo

Hai hình biểu đồ được sinh từ chính các tệp JSON ở trên, không vẽ tay. Script cần `matplotlib`, thư viện này cố ý không nằm trong `requirements.txt` của ứng dụng vì chỉ phục vụ việc soạn báo cáo.

Listing A.7: Sinh lại hình và biên dịch báo cáo

```
python -m pip install matplotlib
python scripts\make_report_figures.py
cd report
latexmk -xelatex main.tex
```

Bảng màu của biểu đồ dùng ba màu phân loại đã được kiểm định về độ tương phản và khả năng phân biệt với người mù màu; mỗi cột đều có nhãn số trực tiếp để không phụ thuộc hoàn toàn vào màu sắc khi in đen trắng.

A.7 Ma trận trường hợp kết thúc trò chơi

Bảng A.2: Thứ tự ưu tiên các điều kiện trong một lượt

Bước	Điều kiện	Kết quả
1	Người trốn đến nút đang có đội máy	`caught`, kể cả nút là lối thoát
2	Người trốn đến lối thoát trống	`escaped`, máy không đi tiếp
3	Máy đi đến nút người trốn	`caught`
4	Đạt giới hạn lượt và chưa thoát/bị bắt	`timeout`
5	Không điều kiện nào đúng	`playing`

A.8 Định nghĩa ký hiệu

Bảng A.3: Ký hiệu toán học sử dụng trong báo cáo

Ký hiệu	Ý nghĩa
$G = (V, E, w)$	Đa đồ thị có hướng và hàm trọng số dương
$d(u, v)$	Chi phí đường đi ngắn nhất từ u tới v
D_{ij}	Chi phí giữa hai điểm quan tâm i và j
$F(S, j)$	Chi phí Held–Karp nhỏ nhất ghé tập S và kết thúc tại j
r_t	Vị trí thật của người trốn ở lượt t
H_t	Bộ vị trí của các đội truy tìm
X	Tập lối thoát
B_t	Tập vị trí người trốn có thể đang ở theo quan sát công khai
b_r, b_h	Ngân sách di chuyển của người trốn và mỗi đội máy
$J(M)$	Hàm điểm phối hợp của tập nước đi máy M

TÀI LIỆU THAM KHẢO

[1] D. Delling, P. Sanders, D. Schultes, and D. Wagner, "Engineering route planning algorithms," *Algorithmics of Large and Complex Networks*, vol. 5515, pp. 117–139, 2009.

[2] P. Toth and D. Vigo, "Vehicle routing: Problems, methods, and applications," *MOS-SIAM Series on Optimization*, 2014.

[3] OpenStreetMap contributors, "Openstreetmap." `https://www.openstreetmap.org/`. Dữ liệu truy cập và đóng gói trong dự án năm 2026.

[4] G. Boeing, "OSMnx: A python package to work with graph-theoretic openstreetmap street networks," *Journal of Open Source Software*, vol. 2, no. 12, p. 215, 2017.

[5] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA: MIT Press, 3 ed., 2009.

[7] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.

[8] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *Journal of the Society for Industrial and Applied Mathematics*, vol. 10, no. 1, pp. 196–210, 1962.

[9] R. Bellman, "Dynamic programming treatment of the travelling salesman problem," *Journal of the ACM*, vol. 9, no. 1, pp. 61–63, 1962.

[10] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, "An analysis of several heuristics for the traveling salesman problem," *SIAM Journal on Computing*, vol. 6, no. 3, pp. 563–581, 1977.

[11] G. A. Croes, "A method for solving traveling-salesman problems," *Operations Research*, vol. 6, no. 6, pp. 791–812, 1958.

[12] T. D. Parsons, “Pursuit-evasion in a graph,” in *Theory and Applications of Graphs*, vol. 642 of *Lecture Notes in Mathematics*, pp. 426–441, Springer, 1978.

[13] T. H. Chung, G. A. Hollinger, and V. Isler, “Search and pursuit-evasion in mobile robotics: A survey,” *Autonomous Robots*, vol. 31, no. 4, pp. 299–316, 2011.

[14] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.

[15] NetworkX developers, “Networkx documentation.” `https://networkx.org/documentation/stable/`, 2026.

[16] Streamlit Inc., “Streamlit documentation.” `https://docs.streamlit.io/`, 2026.

[17] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, “A survey of monte carlo tree search methods,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.